Release 1 : Prise de Rendez-vous

Authentification & Gestion des Rendez-vous

MediConnect — Plateforme e-Santé

NestJS + Angular 17 • PFE 2026

2025 – 2026

3.1 Introduction

Ce chapitre se concentre sur les étapes de réalisation de notre deuxième release en mettant l'accent sur la création et le raffinement du backlog sprint 0 et sprint 1, la conception et la réalisation.

Le sprint 0 couvre les fonctionnalités d'authentification et d'inscription pour l'ensemble des acteurs de la plateforme e-Santé (patients, médecins, pharmaciens et administrateur). Le sprint 1 se concentre sur la prise de rendez-vous médicaux, incluant la recherche de médecins, la sélection de créneaux en temps réel, les confirmations et rappels automatiques.

3.2 Backlog de Release 1

Pour planifier les tâches à accomplir, nous allons créer le backlog du release 2 présenté sur le tableau 3.1. L'estimation est utilisée pour évaluer la charge de travail associée à chaque tâche.

User Story (Backlog Produit)	Priorité	Évaluation	Sprint	Acteur	Workflow
Sprint 1 — *Authentification & Inscription*					
En tant qu'administrateur, je veux m'authentifier pour accéder au back-office	1	Moyen	Sprint 0	Admin	Auth
En tant que patient, je veux m'inscrire (nom, email, mot de passe, profil médical de base)	1	Moyen	Sprint 0	Patient	Auth
En tant que patient, je veux m'authentifier	1	Moyen	Sprint 0	Patient	Auth
En tant que patient, je veux réinitialiser mon mot de passe par email	1	Faible	Sprint 0	Patient	Auth
En tant que médecin, je veux m'inscrire (RPPS, spécialité, cabinet)	1	Moyen	Sprint 0	Médecin	Auth
En tant que médecin, je veux m'authentifier	1	Moyen	Sprint 0	Médecin	Auth
En tant que médecin, je veux réinitialiser mon mot de passe	1	Faible	Sprint 0	Médecin	Auth
En tant que pharmacien, je veux m'inscrire (numéro ordre, officine)	1	Moyen	Sprint 0	Pharmacien	Auth
En tant que pharmacien, je veux m'authentifier	1	Moyen	Sprint 0	Pharmacien	Auth
Sprint 2 — *W1 — Prise de Rendez-vous*					
En tant que patient, je veux rechercher un médecin par spécialité, localisation et disponibilité	2	Fort	Sprint 1	Patient	W1 RDV
En tant que patient, je veux consulter le profil d'un médecin (photo, avis, tarifs)	2	Moyen	Sprint 1	Patient	W1 RDV
En tant que patient, je veux sélectionner un créneau disponible en temps réel (physique ou téléconsultation)	2	Fort	Sprint 1	Patient	W1 RDV
En tant que médecin, je veux recevoir une notification instantanée lors d'une demande de RDV	2	Fort	Sprint 1	Médecin	W1 RDV

User Story (Backlog Produit)	Priorité	Évaluation	Sprint	Acteur	Workflow
En tant que médecin, je veux accepter ou refuser une demande de RDV en 1 clic (statut VALIDÉ / REFUSÉ)	2	Fort	Sprint 1	Médecin	W1 RDV
En tant que patient, je veux recevoir une confirmation de RDV par email et SMS	2	Moyen	Sprint 1	Patient	W1 RDV
En tant que patient, je veux recevoir un rappel automatique 24h avant le RDV avec le lien de téléconsultation	2	Moyen	Sprint 1	Patient	W1 RDV
En tant que patient, je veux annuler ou reporter un RDV depuis mon espace	2	Moyen	Sprint 1	Patient	W1 RDV

Tableau 3.1 – Backlog de Release 1

3.3 Raffinement de Release 1

Dans cette section nous examinons ces cas d'utilisation :

- S'inscrire
- S'authentifier
- Prendre un rendez-vous
- Gérer les rendez-vous

3.3.1 Raffinement de Sprint 0

Raffinement du cas "S'inscrire"

La figure 3.1 offre une représentation visuelle du cas "S'inscrire", décrivant le processus que les utilisateurs suivent pour s'enregistrer sur la plateforme e-Santé :

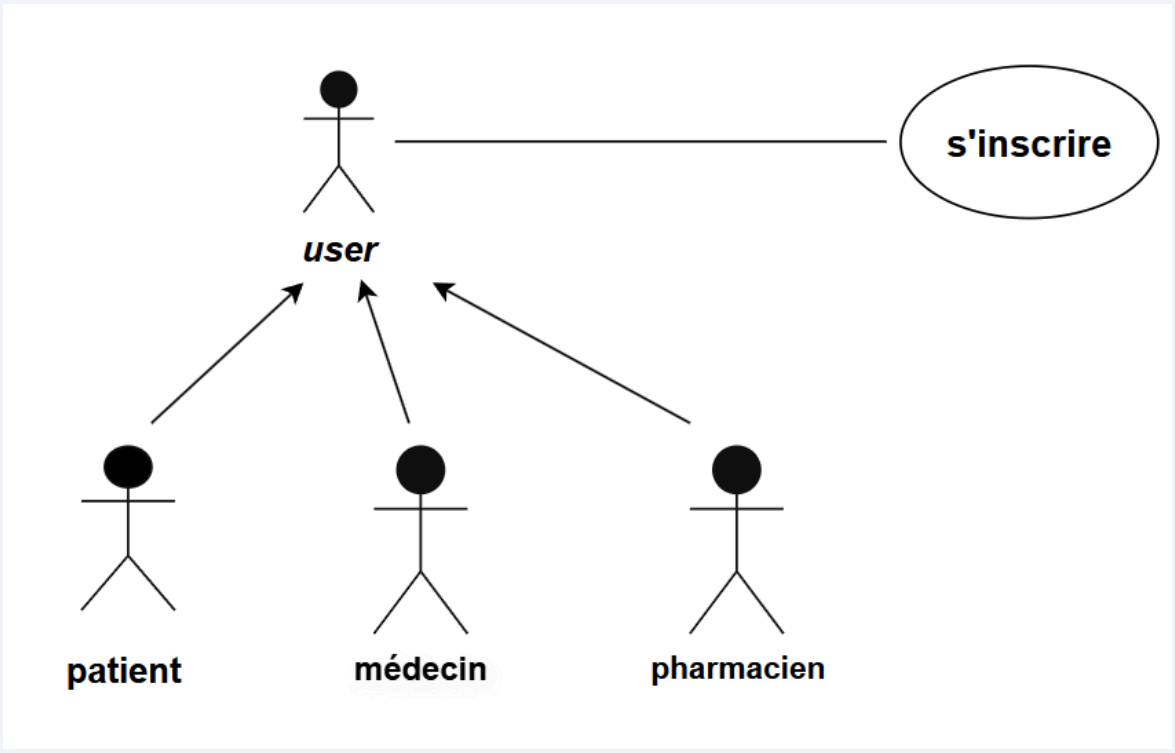


Figure 3.1 – Décomposition de cas d'utilisation « S'inscrire »

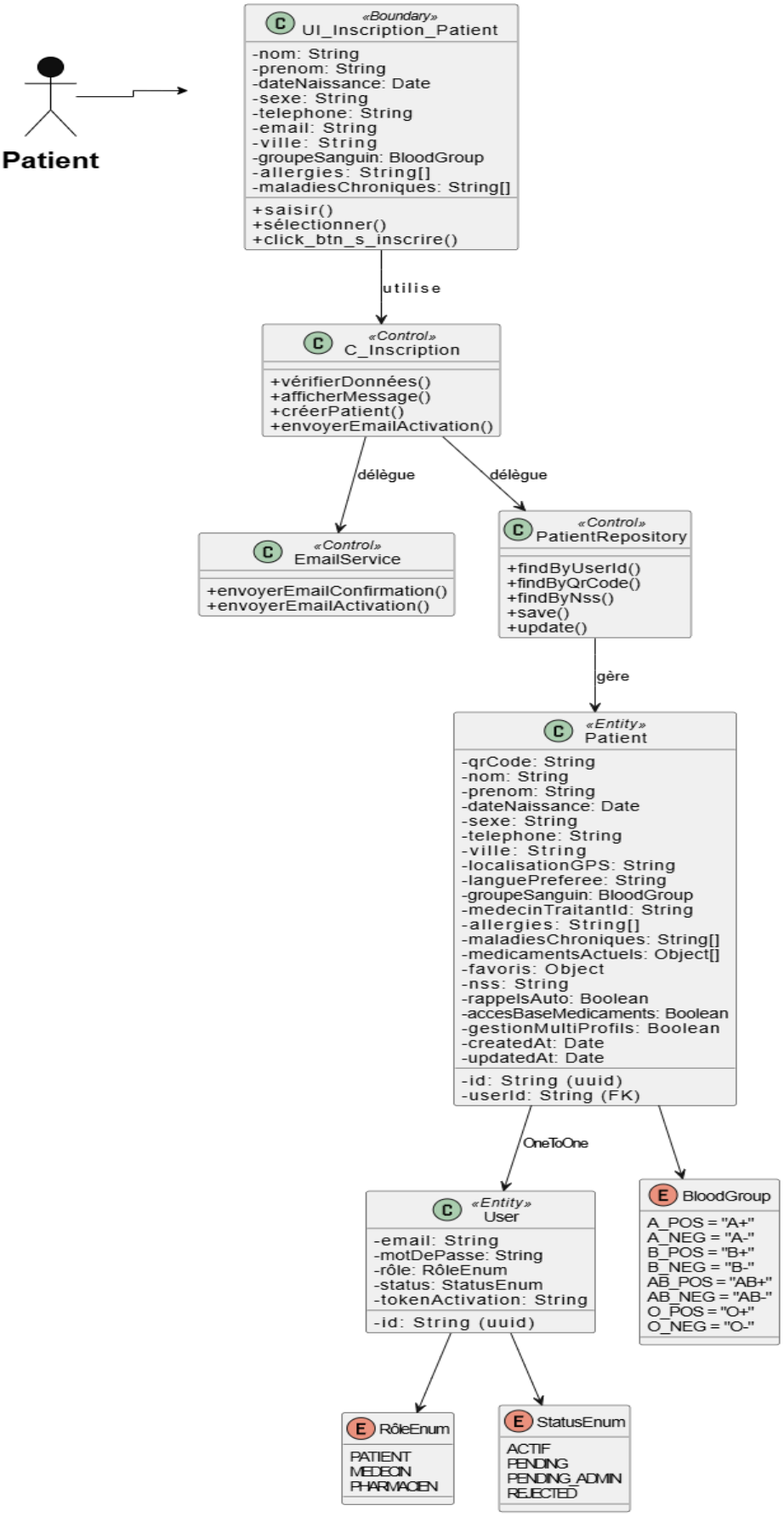


Figure 3.3 –Diagramme de classe Analytique du cas « S'inscrire patient »

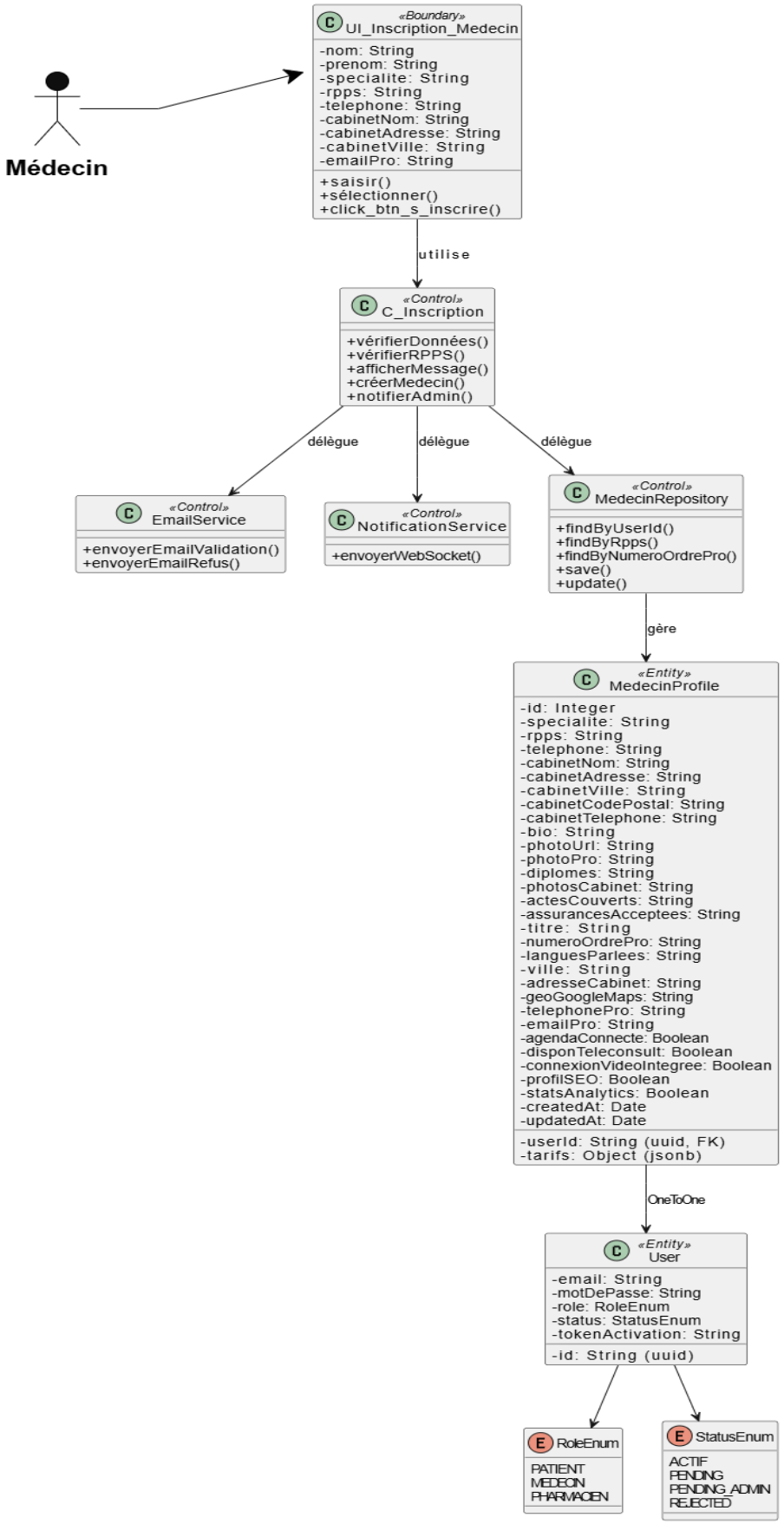


Figure 3.4 –Diagramme de classe Analytique du cas « S'inscrire médecin»

Dans cette figure 3.2 on trouve le diagramme de séquence associé au cas "S'inscrire" :

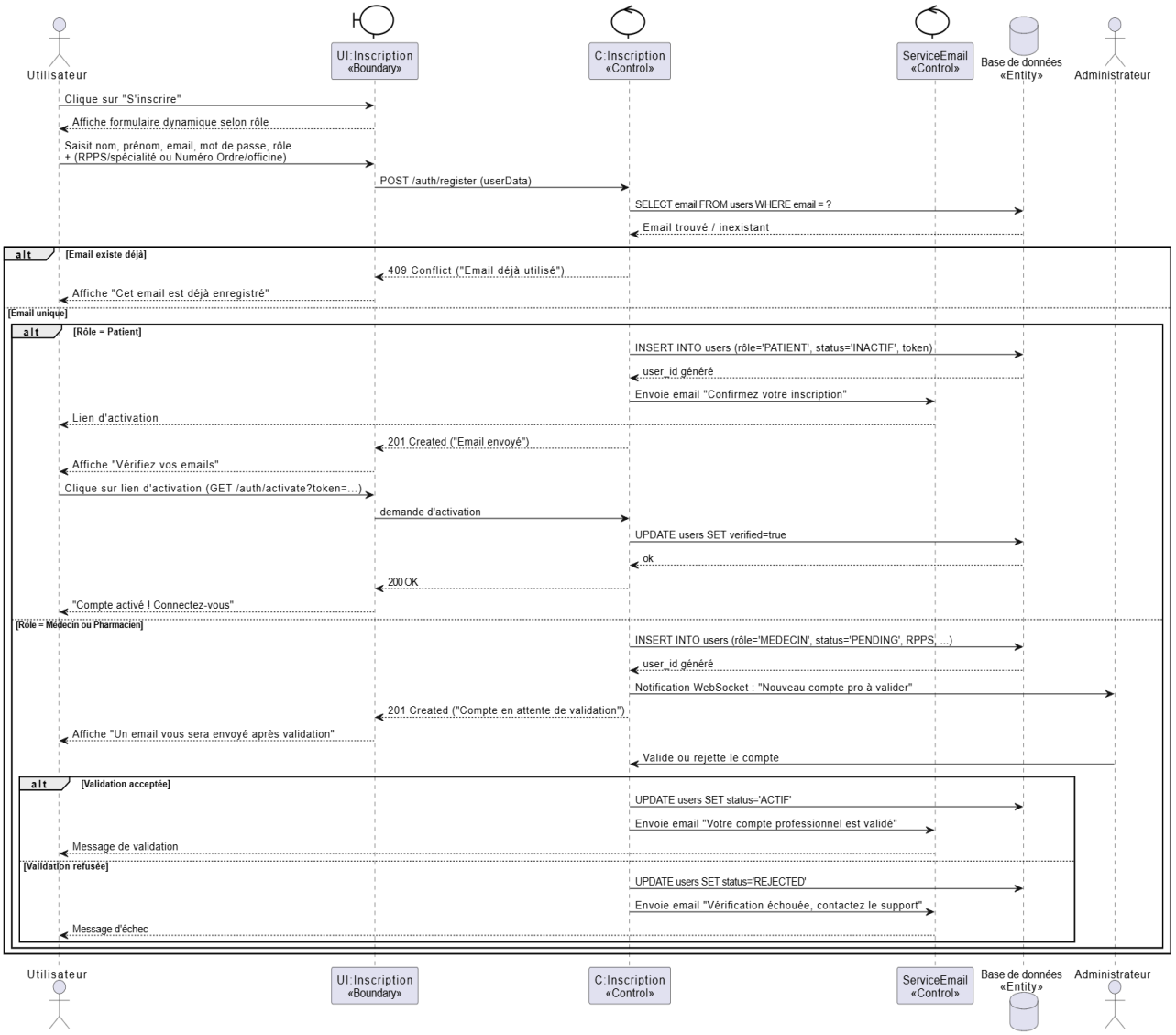
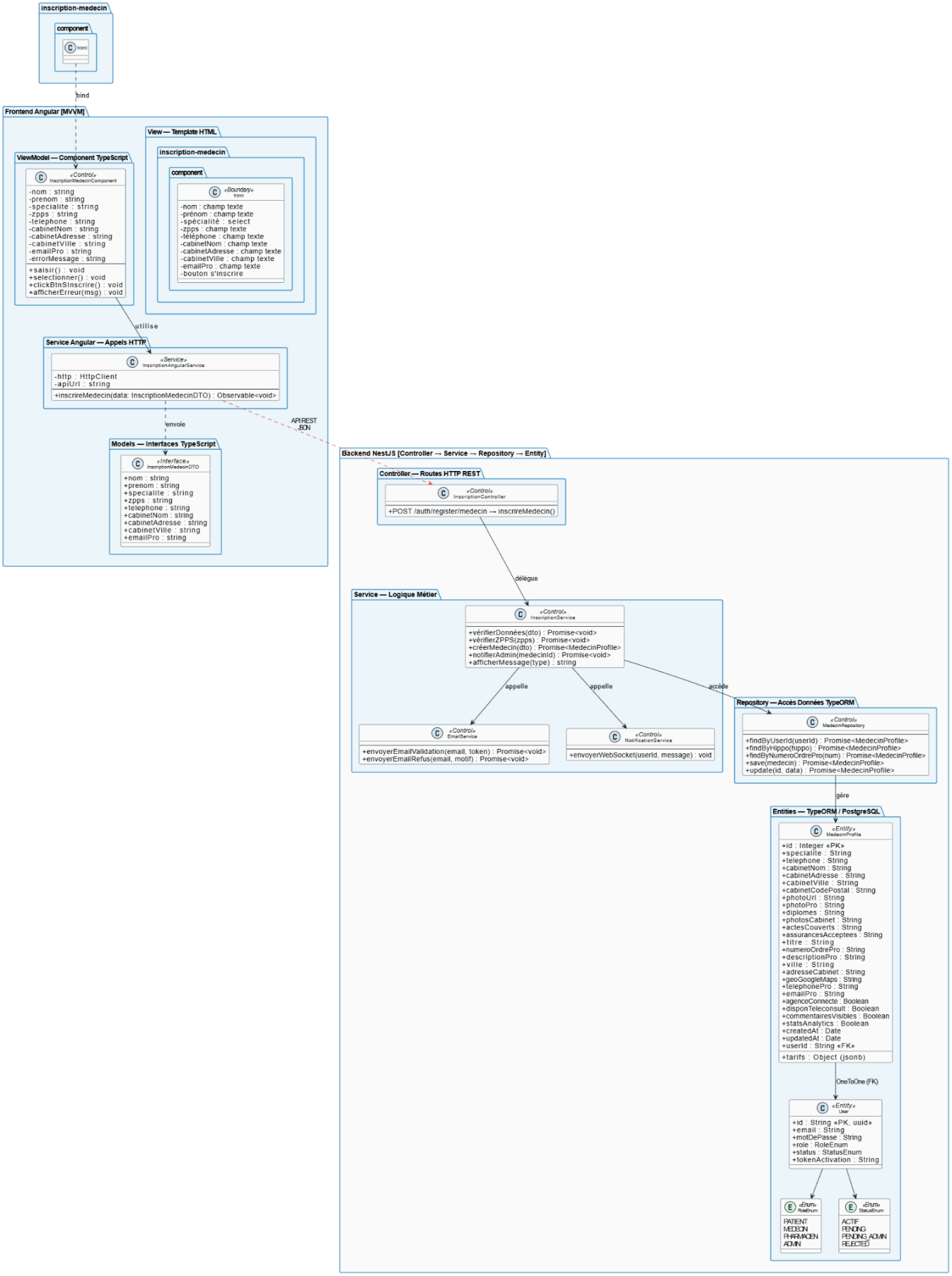
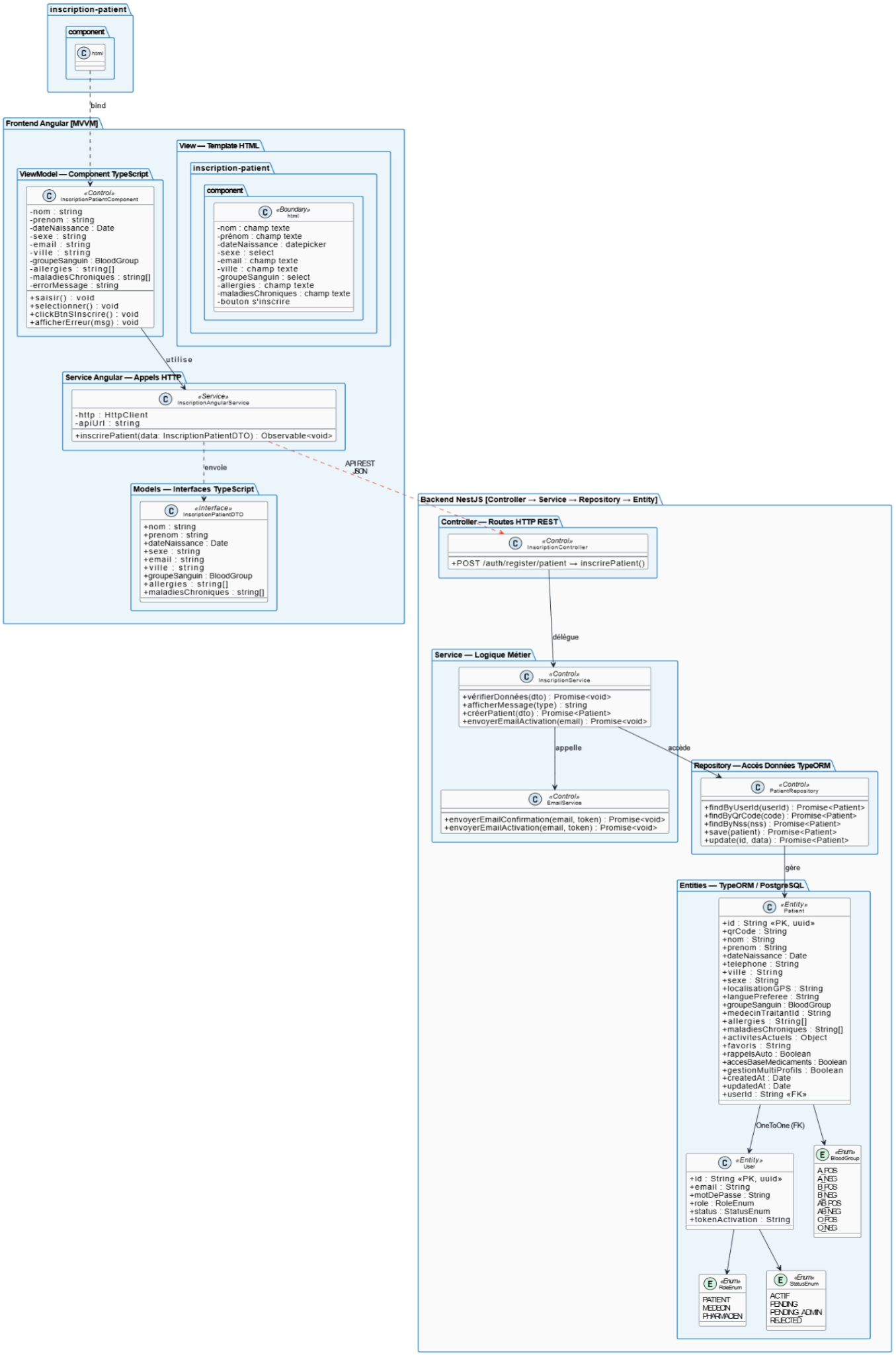


Figure 3.2 –Diagramme de séquence Analytique du cas « S'inscrire »


inscription-medecin
component
html
bind
Frontend Angular [MVVM]
View — Template HTML
inscription-medecin
component
«Boundary»
html
-nom : champ texte
-prénom : champ texte
-spécialité : select
-zpps : champ texte
-téléphone : champ texte
-cabinetNom : champ texte
-cabinetAdresse : champ texte
-cabinetVille : champ texte
-emailPro : champ texte
-bouton s'inscrire
ViewModel — Component TypeScript
«Control»
InscriptionMedecinComponent
-nom : string
-prenom : string
-specialite : string
-zpps : string
-telephone : string
-cabinetNom : string
-cabinetAdresse : string
-cabinetVille : string
-emailPro : string
-errorMessage : string
+saisir() : void
+selectionner() : void
+clickBtnSInscrire() : void
+afficherErreur(msg) : void
utilise
Service Angular — Appels HTTP
«Service»
InscriptionAngularService
-http : HttpClient
-apiUrl : string
+inscrireMedecin(data: InscriptionMedecinDTO) : Observable<void>
envoie
API REST JSON
Models — Interfaces TypeScript
«Interface»
InscriptionMedecinDTO
+nom : string
+prenom : string
+specialite : string
+zpps : string
+telephone : string
+cabinetNom : string
+cabinetAdresse : string
+cabinetVille : string
+emailPro : string
Backend NestJS [Controller → Service → Repository → Entity]
Contrôller — Routes HTTP REST
«Control»
InscriptionController
+POST /auth/register/medecin → inscrireMedecin()
délégue
Service — Logique Métier
«Control»
InscriptionService
+vérifierDonnées(dto) : Promise<void>
+vérifierZPPS(zpps) : Promise<void>
+créerMedecin(dto) : Promise<MedecinProfile>
+notifierAdmin(medecinId) : Promise<void>
+afficherMessage(type) : string
appelle
appelle
accède
«Control»
EmailService
+envoyerEmailValidation(email, token) : Promise<void>
+envoyerEmailRefus(email, motif) : Promise<void>
«Control»
NotificationService
+envoyerWebSocket(userId, message) : void
Repository — Accès Données TypeORM
«Control»
MedecinRepository
+findByUserId(userId) : Promise<MedecinProfile>
+findByHippo(hippo) : Promise<MedecinProfile>
+findByNumeroOrdrePro(num) : Promise<MedecinProfile>
+save(medecin) : Promise<MedecinProfile>
+update(id, data) : Promise<MedecinProfile>
gère
Entities — TypeORM / PostgreSQL
«Entity»
MedecinProfile
+id : Integer «PK»
+specialite : String
+telephone : String
+cabinetNom : String
+cabinetAdresse : String
+cabinetVille : String
+cabinetCodePostal : String
+photoUrl : String
+photoPro : String
+diplomes : String
+photosCabinet : String
+actesCouverts : String
+assurancesAcceptees : String
+titre : String
+numeroOrdrePro : String
+descriptionPro : String
+ville : String
+adresseCabinet : String
+geoGoogleMaps : String
+telephonePro : String
+emailPro : String
+agenceConnecte : Boolean
+disponTeleconsult : Boolean
+commentairesVisibles : Boolean
+statsAnalytics : Boolean
+createdAt : Date
+updatedAt : Date
+userId : String «FK»
+tarifs : Object (jsonb)
OneToOne (FK)
«Entity»
User
+id : String «PK, uuid»
+email : String
+motDePasse : String
+role : RoleEnum
+status : StatusEnum
+tokenActivation : String
«Enum»
RoleEnum
PATIENT
MEDECIN
PHARMACIEN
ADMIN
«Enum»
StatusEnum
ACTIF
PENDING
PENDING_ADMIN
REJECTED


inscription-patient
component
html
bind
Frontend Angular [MVVM]
View — Template HTML
inscription-patient
component
«Boundary»
html
-nom : champ texte
-prénom : champ texte
-dateNaissance : datepicker
-sexe : select
-email : champ texte
-ville : champ texte
-groupeSanguin : select
-allergies : champ texte
-maladiesChroniques : champ texte
-bouton s'inscrire
ViewModel — Component TypeScript
«Control»
InscriptionPatientComponent
-nom : string
-prenom : string
-dateNaissance : Date
-sexe : string
-email : string
-ville : string
-groupeSanguin : BloodGroup
-allergies : string[]
-maladiesChroniques : string[]
-errorMessage : string
+saisir() : void
+selectionner() : void
+clickBtnSInscrire() : void
+afficherErreur(msg) : void
utilise
Service Angular — Appels HTTP
«Service»
InscriptionAngularService
-http : HttpClient
-apiUrl : string
+inscrirePatient(data: InscriptionPatientDTO) : Observable<void>
envoie
API REST
JSON
Models — Interfaces TypeScript
«Interface»
InscriptionPatientDTO
+nom : string
+prenom : string
+dateNaissance : Date
+sexe : string
+email : string
+ville : string
+groupeSanguin : BloodGroup
+allergies : string[]
+maladiesChroniques : string[]
Backend NestJS [Controller → Service → Repository → Entity]
Controller — Routes HTTP REST
«Control»
InscriptionController
+POST /auth/register/patient → inscrirePatient()
délègue
Service — Logique Métier
«Control»
InscriptionService
+vérifierDonnées(dto) : Promise<void>
+afficherMessage(type) : string
+créerPatient(dto) : Promise<Patient>
+envoyerEmailActivation(email) : Promise<void>
appelle
accède
«Control»
EmailService
+envoyerEmailConfirmation(email, token) : Promise<void>
+envoyerEmailActivation(email, token) : Promise<void>
Repository — Accès Données TypeORM
«Control»
PatientRepository
+findByUserId(userId) : Promise<Patient>
+findByQrCode(code) : Promise<Patient>
+findByNss(nss) : Promise<Patient>
+save(patient) : Promise<Patient>
+update(id, data) : Promise<Patient>
gère
Entities — TypeORM / PostgreSQL
«Entity»
Patient
+id : String «PK, uuid»
+qrCode : String
+nom : String
+prenom : String
+dateNaissance : Date
+telephone : String
+ville : String
+sexe : String
+localisationGPS : String
+languePreferee : String
+groupeSanguin : BloodGroup
+medecinTraitantId : String
+allergies : String[]
+maladiesChroniques : String[]
+activitesActuels : Object
+favoris : String
+rappelsAuto : Boolean
+accesBaseMedicaments : Boolean
+gestionMultiProfils : Boolean
+createdAt : Date
+updatedAt : Date
+userId : String «FK»
OneToOne (FK)
«Entity»
User
+id : String «PK, uuid»
+email : String
+motDePasse : String
+role : RoleEnum
+status : StatusEnum
+tokenActivation : String
«Enum»
BloodGroup
A_POS
A_NEG
B_POS
B_NEG
AB_POS
AB_NEG
O_POS
O_NEG
«Enum»
RoleEnum
PATIENT
MEDECIN
PHARMACIEN
«Enum»
StatusEnum
ACTIF
PENDING
PENDING_ADMIN
REJECTED

Le tableau 3.2 illustre en détail le processus de décomposition du cas "S'inscrire" :

Cas	S'inscrire
Acteur	-Patient -Médecin -Pharmacien
Condition préalable	La plateforme est opérationnelle. Adresse mail et mot de passe valides.
Condition résultante	Inscription réussie.
Description de scénario	- La plateforme affiche une page d'inscription. - L'utilisateur remplit ses informations personnelles. - L'utilisateur appuie sur le bouton « S'inscrire ». - La plateforme inspecte les informations. - La plateforme indique un avis de succès.
Scénarii	- La plateforme indique une imprécision si les informations sont inexactes.

Tableau 3.4 – Décomposition du cas « S'inscrire »

Raffinement du cas "S'authentifier"

La figure 3.2 démontre une décomposition de cas d'utilisation "S'authentifier" :

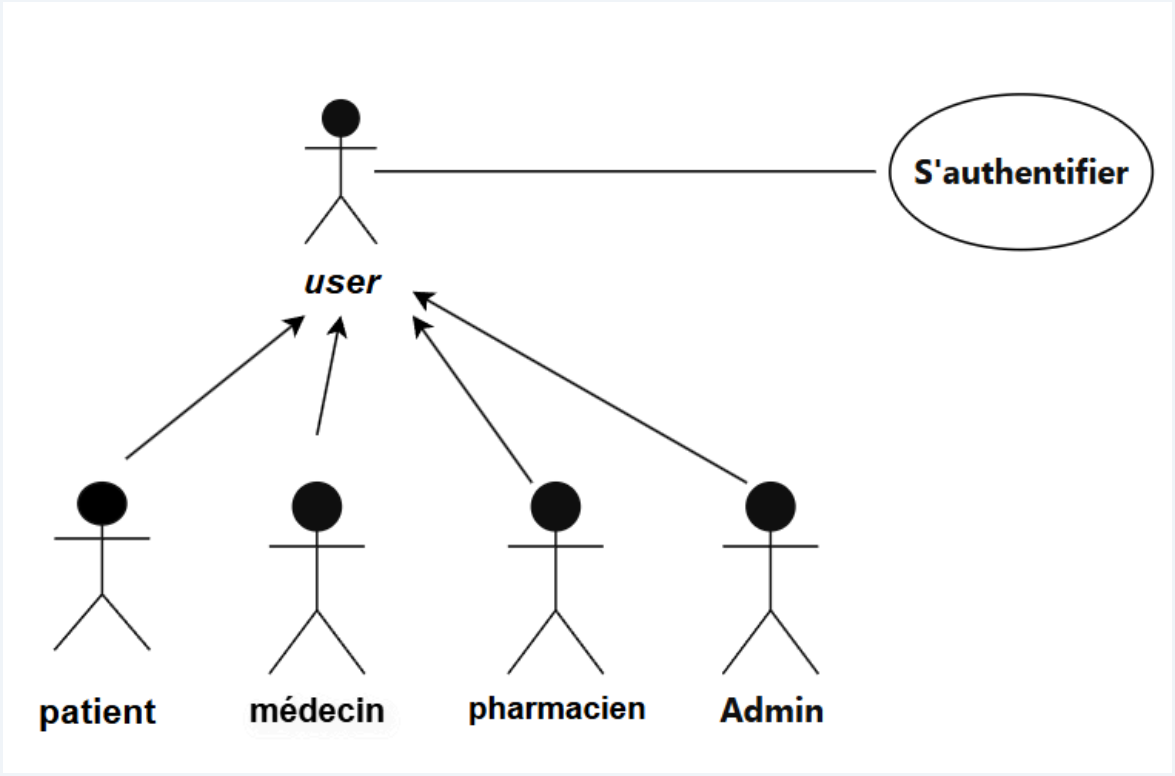


Figure 3.2 – Raffinement du cas d'utilisation « S'authentifier »

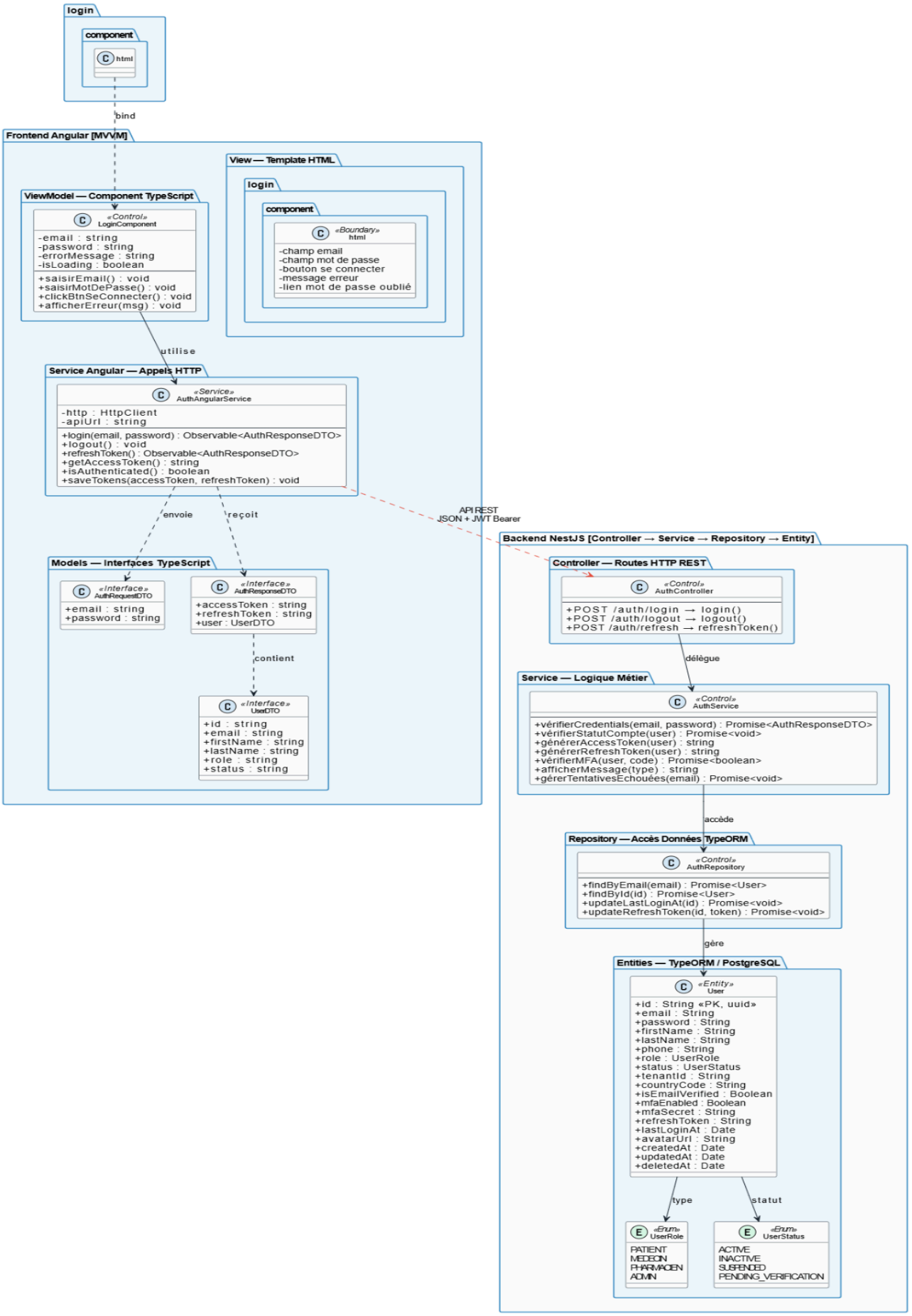


Figure 3.2 –Diagramme de classe Analytique du cas « S'authentifier »

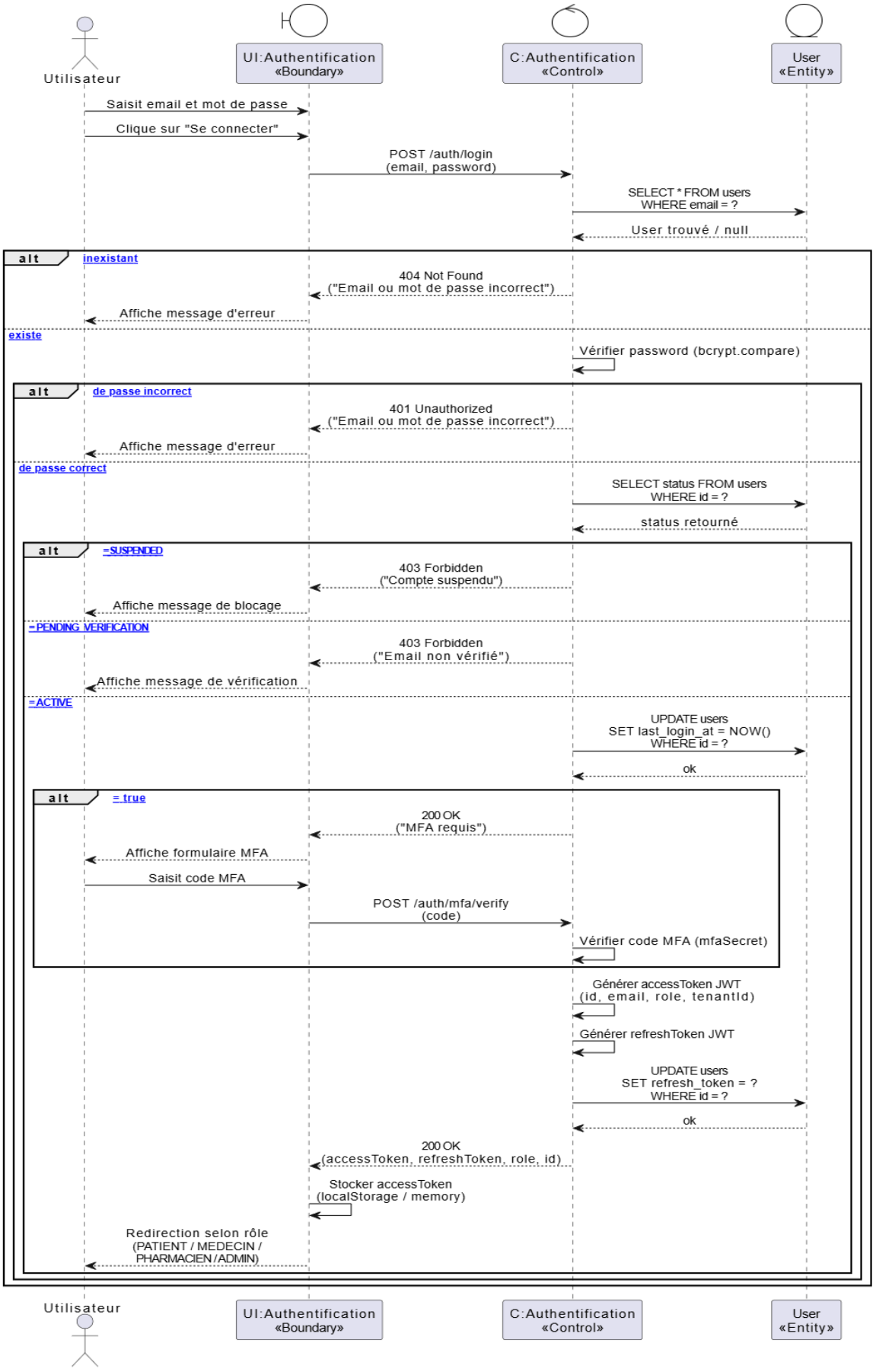


Figure 3.2 –Diagramme de séquence Analytique du cas « S'authentifier »

Le Tableau 3.3 illustre la décomposition détaillée du cas d'utilisation "S'authentifier" :

Cas	S'authentifier
Acteur	-Administrateur -Médecin -Patient -Pharmacien
Condition préalable	La plateforme est opérationnelle.
Condition résultante	Utilisateur authentifié.
Description de scénario	- La plateforme affiche la page d'authentification. - L'utilisateur saisit son adresse e-mail et son mot de passe. - La plateforme inspecte l'adresse e-mail et le mot de passe. - La plateforme oriente vers la page suivante.
Scénarii	- La plateforme indique une imprécision si les informations sont inexactes.

Tableau 3.3 – Raffinement du cas « S'authentifier »

3.3.2 Raffinement de Sprint 1

Raffinement du cas "Prendre un rendez-vous"

La figure 3.3 présente la décomposition détaillée du cas "Prendre un rendez-vous" :

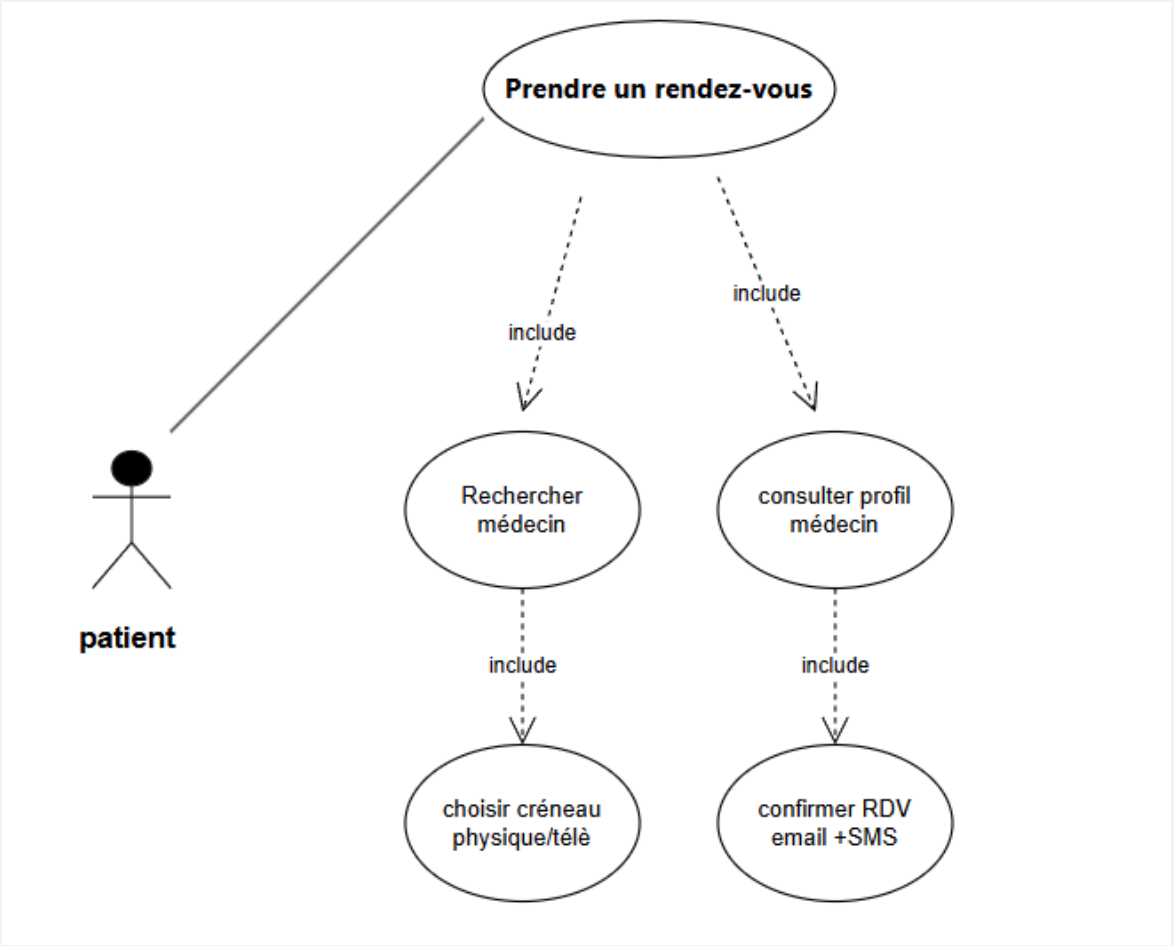


Figure 3.3 – Décomposition de cas d'utilisation « Prendre un rendez-vous »

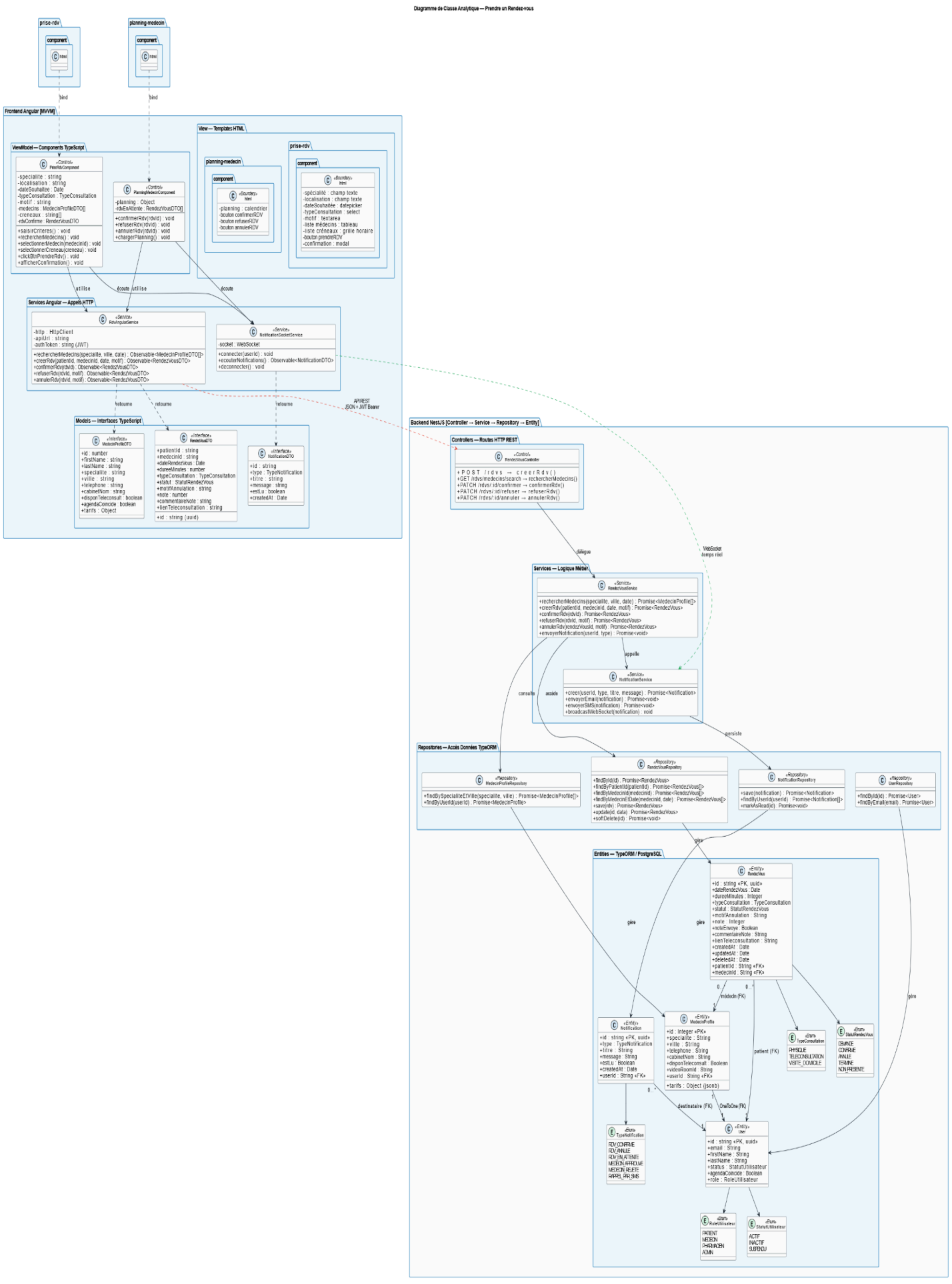

Diagramme de Classe Analytique — Prendre un Rendez-vous
prise-rdv
planning-medecin
component
bind
Frontend Angular [MVVM]
View — Templates HTML
ViewModel — Components TypeScript
PriseRdvComponent
-specialite : string
-localisation : string
-dateSouhaitee : Date
-typeConsultation : TypeConsultation
-motif : string
-medecins : MedecinProfileDTO[]
-creneaux : string[]
-rdvConfirme : RendezVousDTO
+saisirCriteres() : void
+rechercherMedecins() : void
+selectionnerMedecin(medecinId) : void
+selectionnerCreneau(creneau) : void
+clickBtnPrendreRdv() : void
+afficherConfirmation() : void
PlanningMedecinComponent
-planning : Object
-rdvEnAttente : RendezVousDTO[]
+confirmerRdv(rdvId) : void
+refuserRdv(rdvId) : void
+annulerRdv(rdvId) : void
+chargerPlanning() : void
-planning : calendrier
-bouton confirmerRDV
-bouton refuserRDV
-bouton annulerRDV
-spécialité : champ texte
-localisation : champ texte
-dateSouhaitée : datepicker
-typeConsultation : select
-motif : textarea
-liste médecins : tableau
-liste créneaux : grille horaire
-bouton prendreRDV
-confirmation : modal
utilise
écoute
Services Angular — Appels HTTP
RdvAngularService
-http : HttpClient
-apiUrl : string
-authToken : string (JWT)
+rechercherMedecins(specialite, ville, date) : Observable<MedecinProfileDTO[]>
+creerRdv(patientId, medecinId, date, motif) : Observable<RendezVousDTO>
+confirmerRdv(rdvId) : Observable<RendezVousDTO>
+refuserRdv(rdvId, motif) : Observable<RendezVousDTO>
+annulerRdv(rdvId, motif) : Observable<RendezVousDTO>
NotificationSocketService
-socket : WebSocket
+connecter(userId) : void
+ecouterNotifications() : Observable<NotificationDTO>
+deconnecter() : void
retourne
API REST
JSON + JWT Bearer
Models — Interfaces TypeScript
MedecinProfileDTO
+id : number
+firstName : string
+lastName : string
+specialite : string
+ville : string
+telephone : string
+cabinetNom : string
+disponTeleconsult : boolean
+agendaCoincide : boolean
+tarifs : Object
RendezVousDTO
+patientId : string
+medecinId : string
+dateRendezVous : Date
+dureeMinutes : number
+typeConsultation : TypeConsultation
+statut : StatutRendezVous
+motifAnnulation : string
+note : number
+commentaireNote : string
+lienTeleconsultation : string
+id : string (uuid)
NotificationDTO
+id : string
+type : TypeNotification
+titre : string
+message : string
+estLu : boolean
+createdAt : Date
Backend NestJS [Controller → Service → Repository → Entity]
Controllers — Routes HTTP REST
RendezVousController
+POST /rdvs → creerRdv()
+GET /rdvs/medecins/search → rechercherMedecins()
+PATCH /rdvs/:id/confirmer → confirmerRdv()
+PATCH /rdvs/:id/refuser → refuserRdv()
+PATCH /rdvs/:id/annuler → annulerRdv()
délègue
WebSocket temps réel
Services — Logique Métier
RendezVousService
+rechercherMedecins(specialite, ville, date) : Promise<MedecinProfile[]>
+creerRdv(patientId, medecinId, date, motif) : Promise<RendezVous>
+confirmerRdv(rdvId) : Promise<RendezVous>
+refuserRdv(rdvId, motif) : Promise<RendezVous>
+annulerRdv(rendezVousId, motif) : Promise<RendezVous>
+envoyerNotification(userId, type) : Promise<void>
appelle
NotificationService
+creer(userId, type, titre, message) : Promise<Notification>
+envoyerEmail(notification) : Promise<void>
+envoyerSMS(notification) : Promise<void>
+broadcastWebSocket(notification) : void
consulte
accède
persiste
Repositories — Accès Données TypeORM
MedecinProfileRepository
+findBySpecialiteEtVille(specialite, ville) : Promise<MedecinProfile[]>
+findByUserId(userId) : Promise<MedecinProfile>
RendezVousRepository
+findById(id) : Promise<RendezVous>
+findByPatientId(patientId) : Promise<RendezVous[]>
+findByMedecinId(medecinId) : Promise<RendezVous[]>
+findByMedecinIdEtDate(medecinId, date) : Promise<RendezVous[]>
+save(rdv) : Promise<RendezVous>
+update(id, data) : Promise<RendezVous>
+softDelete(id) : Promise<void>
NotificationRepository
+save(notification) : Promise<Notification>
+findByUserId(userId) : Promise<Notification[]>
+markAsRead(id) : Promise<void>
UserRepository
+findById(id) : Promise<User>
+findByEmail(email) : Promise<User>
gère
Entités — TypeORM / PostgreSQL
RendezVous
+id : string «PK, uuid»
+dateRendezVous : Date
+dureeMinutes : Integer
+typeConsultation : TypeConsultation
+statut : StatutRendezVous
+motifAnnulation : String
+note : Integer
+noteEnvoye : Boolean
+commentaireNote : String
+lienTeleconsultation : String
+createdAt : Date
+updatedAt : Date
+deletedAt : Date
+patientId : String «FK»
+medecinId : String «FK»
medecin (FK)
patient (FK)
Notification
+id : string «PK, uuid»
+type : TypeNotification
+titre : String
+message : String
+estLu : Boolean
+createdAt : Date
+userId : String «FK»
MedecinProfile
+id : Integer «PK»
+specialite : String
+ville : String
+telephone : String
+cabinetNom : String
+disponTeleconsult : Boolean
+videoRoomId : String
+userId : String «FK»
+tarifs : Object (jsonb)
destinataire (FK)
OneToOne (FK)
TypeConsultation
PHYSIQUE
TELECONSULTATION
VISITE_DOMICILE
StatutRendezVous
DEMANDE
CONFIRME
ANNULE
TERMINE
NON_PRESENTE
TypeNotification
RDV_CONFIRME
RDV_ANNULE
RDV_EN_ATTENTE
MEDECIN_ATTRIBUE
MEDECIN_REJETE
RAPPEL_J4R_SMS
User
+id : string «PK, uuid»
+email : String
+firstName : String
+lastName : String
+status : StatutUtilisateur
+agendaCoincide : Boolean
+role : RoleUtilisateur
RoleUtilisateur
PATIENT
MEDECIN
PHARMACIEN
ADMIN
StatutUtilisateur
ACTIF
INACTIF
SUSPENDU

Figure 3.3 –Diagramme de classe Analytique du cas « Prendre un rendez-vous »

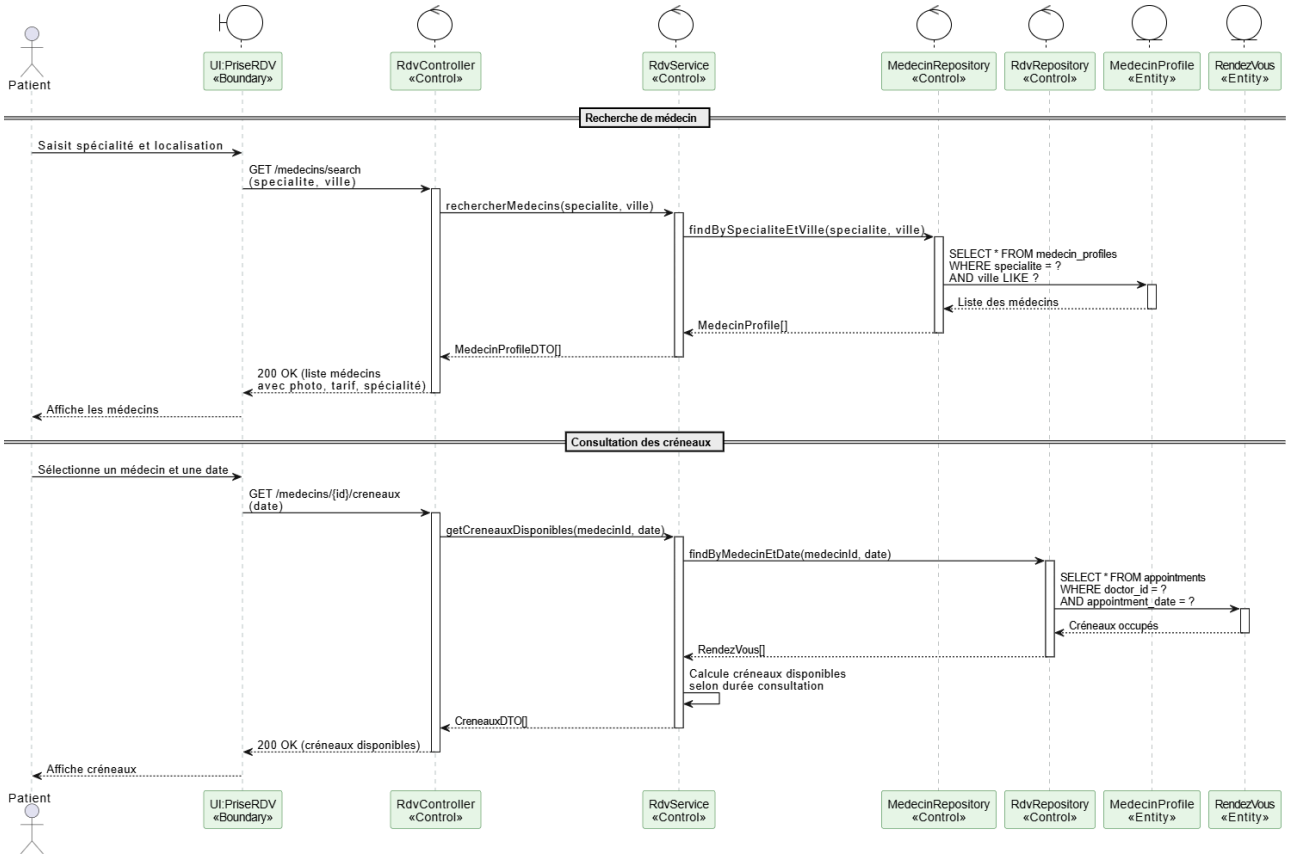


Diagramme de Séquence Analytique — Prendre un RDV (Partie 1)\n[Recherche médecin + Consultation créneaux]

Diagramme de Séquence Analytique — Prendre un RDV (Partie 2)\n[Prise RDV + Médecin accepte ou refuse]

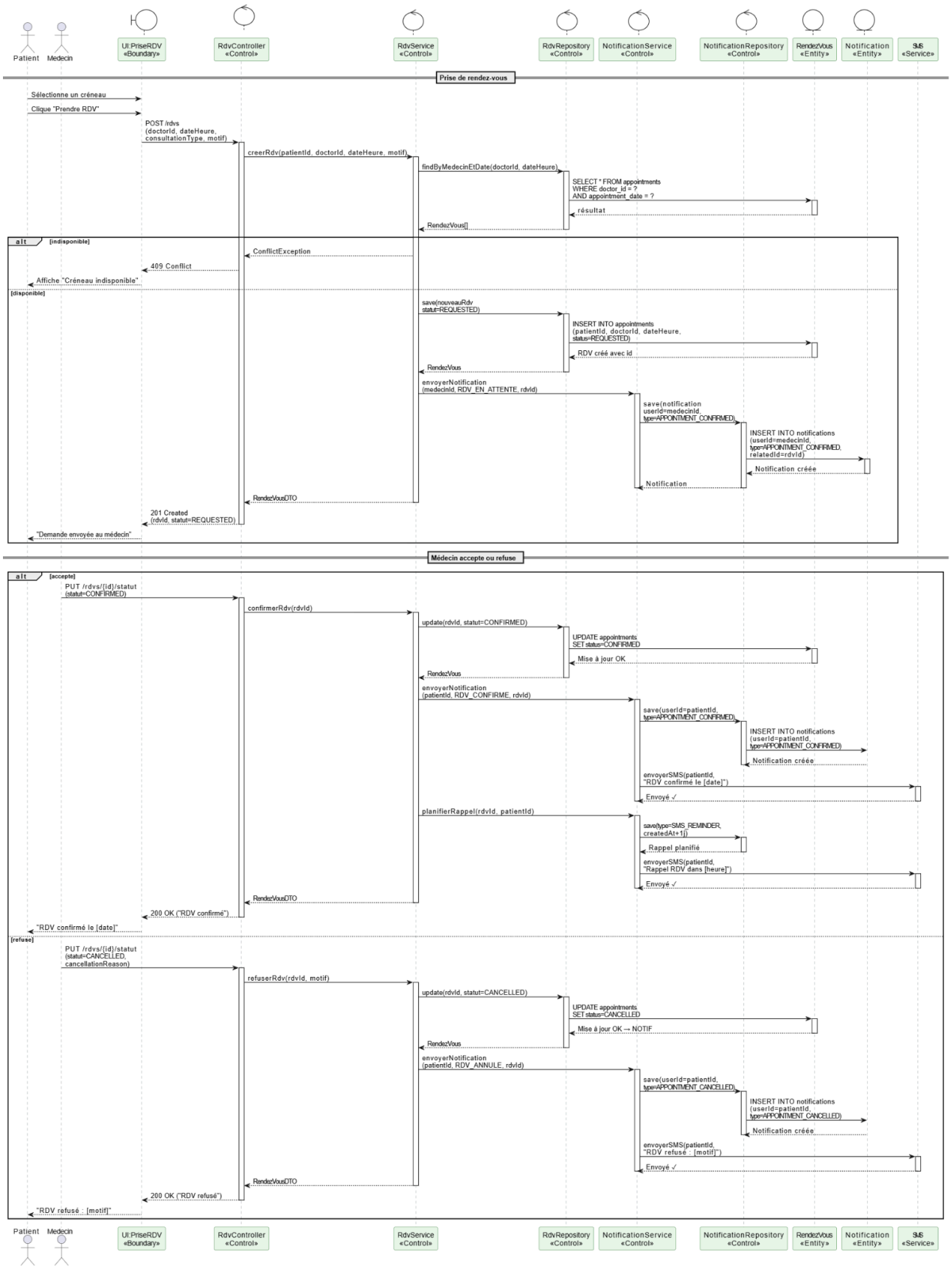


Diagramme de séquence Analytique du cas « Prendre un rendez-vous »

Le Tableau 3.4 présente la décomposition détaillée du cas "Prendre un rendez-vous" :

Cas	Prendre un rendez-vous
Acteur	-Patient
Condition préalable	L'utilisateur est authentifié et la plateforme est opérationnelle.
Condition résultante	Rendez-vous créé et confirmé.
Description de scénario	- Le patient recherche un médecin par spécialité, localisation et disponibilité. - Le patient consulte le profil du médecin (photo, avis, tarifs). - Le patient sélectionne un créneau disponible en temps réel (physique ou téléconsultation). - Le patient confirme la prise de rendez-vous. - La plateforme envoie une confirmation par email et SMS. - Un rappel automatique est envoyé 24h avant le RDV avec le lien de téléconsultation.
Scénarii	- Aucun créneau disponible : la plateforme affiche un message d'indisponibilité.

Tableau 3.4 – Raffinement du cas « Prendre un rendez-vous »

Raffinement du cas "Gérer les rendez-vous"

La figure 3.4 présente le raffinement du cas d'utilisation "Gérer les rendez-vous" :

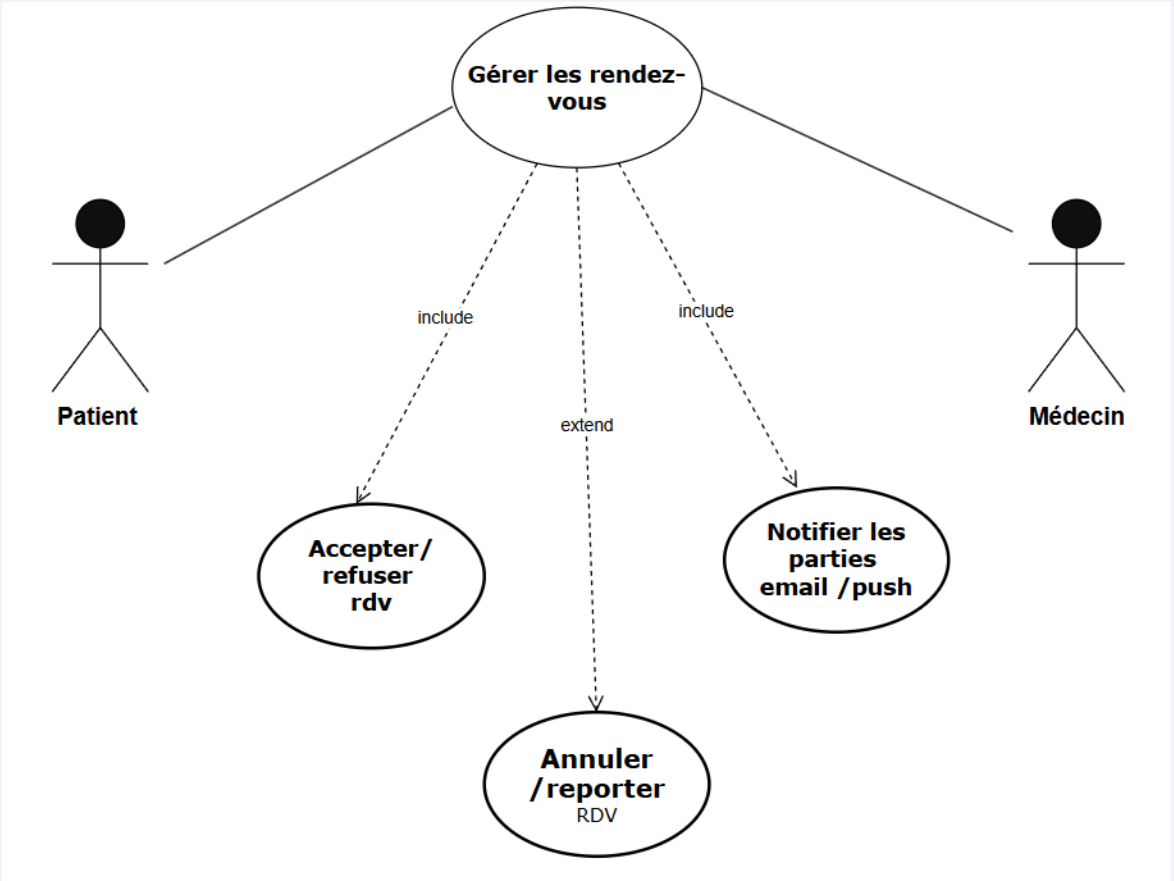


Figure 3.4 – Raffinement du cas « Gérer les rendez-vous »

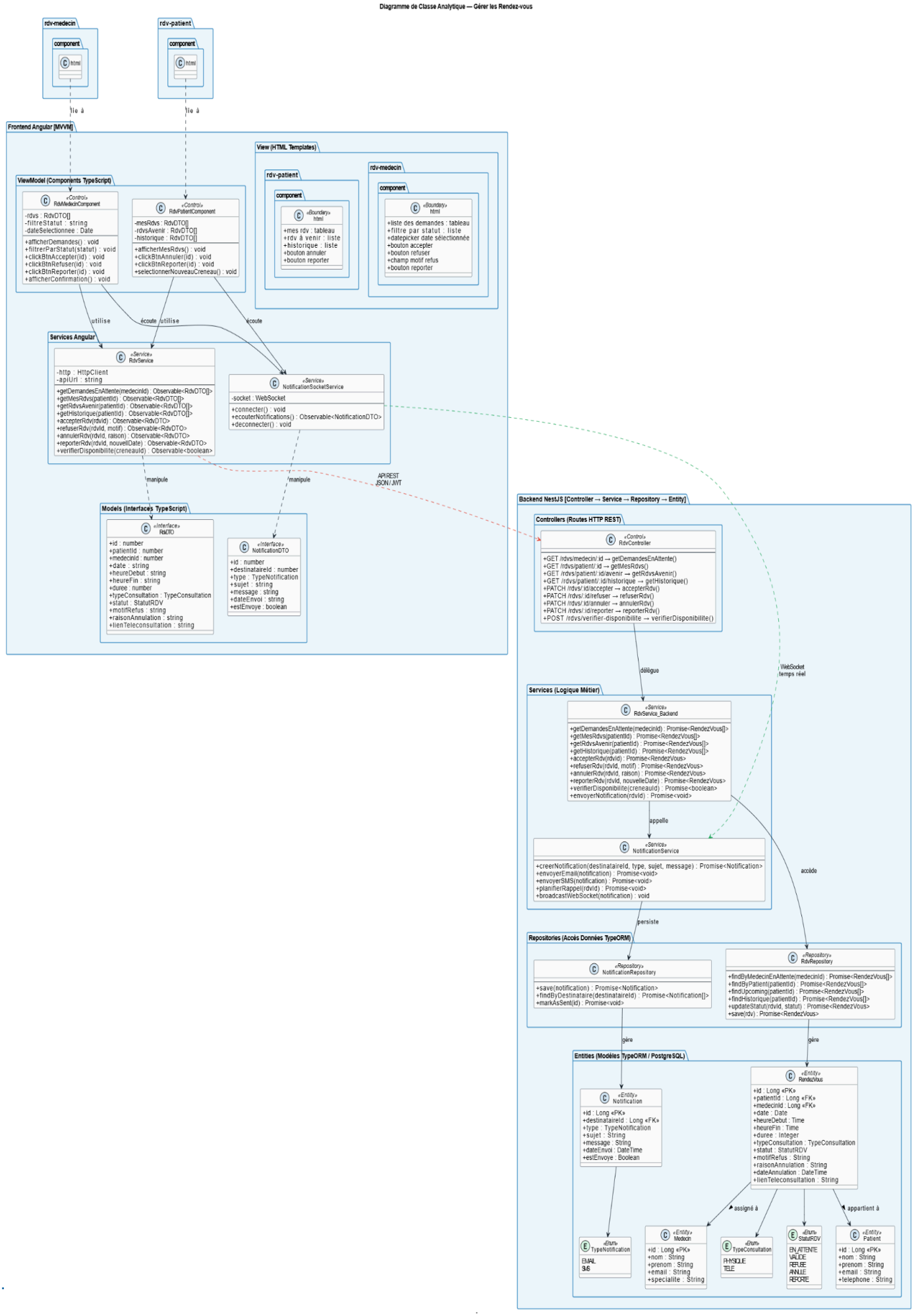
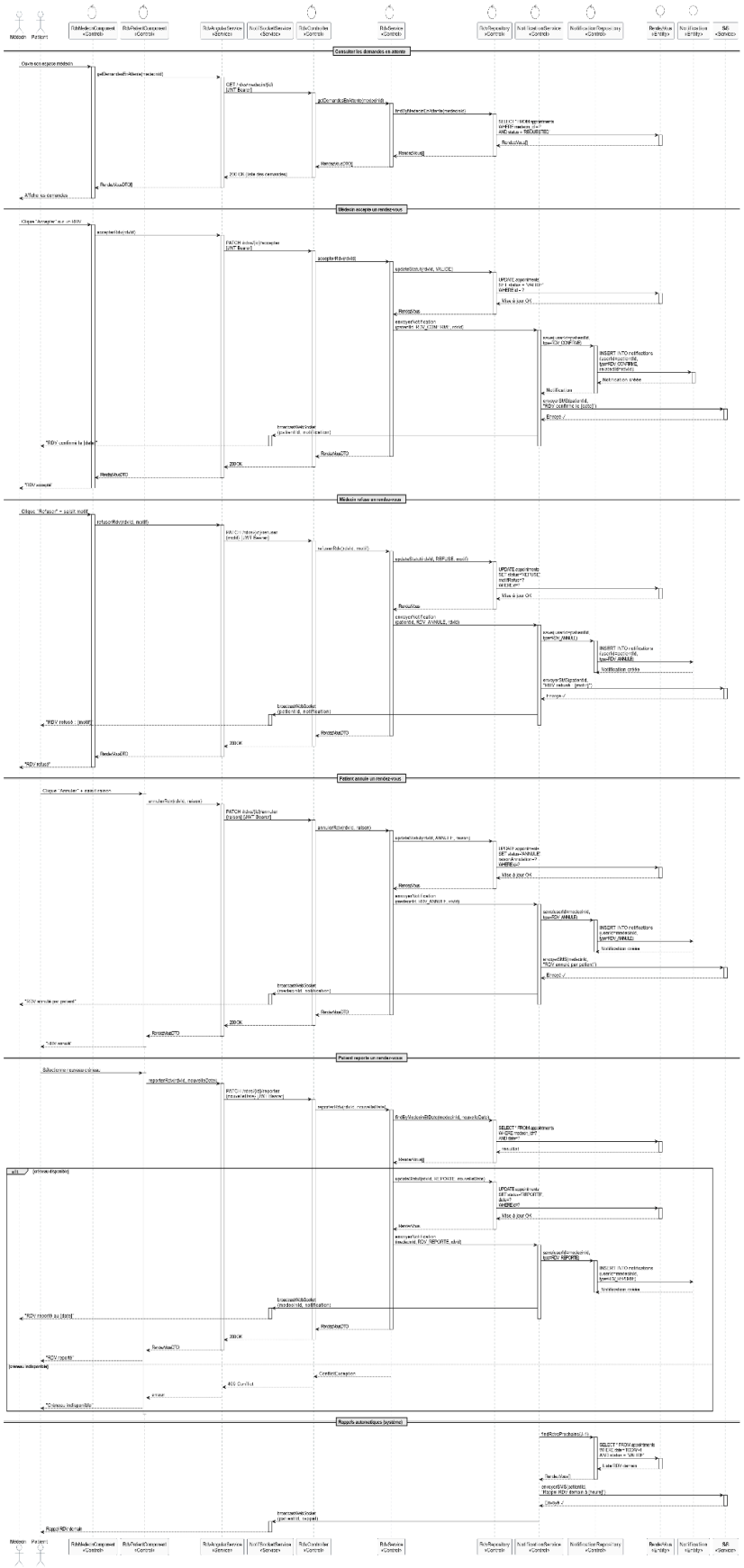

Diagramme de Classe Analytique — Gérer les Rendez-vous
rdv-medecin
component
html
rdv-patient
component
html
lie à
lie à
Frontend Angular [MVVM]
ViewModel (Components TypeScript)
«Control»
RdvMedecinComponent
-rdvs : RdvDTO[]
-filtreStatut : string
-dateSelectionnee : Date
+afficherDemandes() : void
+filtrerParStatut(statut) : void
+clickBtnAccepter(id) : void
+clickBtnRefuser(id) : void
+clickBtnReporter(id) : void
+afficherConfirmation() : void
«Control»
RdvPatientComponent
-mesRdvs : RdvDTO[]
-rdvsAvenir : RdvDTO[]
-historique : RdvDTO[]
+afficherMesRdvs() : void
+clickBtnAnnuler(id) : void
+clickBtnReporter(id) : void
+selectionnerNouveauCreneau() : void
View (HTML Templates)
rdv-patient
component
«Boundary»
html
+mes rdv : tableau
+rdv à venir : liste
+historique : liste
+bouton annuler
+bouton reporter
rdv-medecin
component
«Boundary»
html
+liste des demandes : tableau
+filtre par statut : liste
+datepicker date sélectionnée
+bouton accepter
+bouton refuser
+champ motif refus
+bouton reporter
utilise
écoute
utilise
écoute
Services Angular
«Service»
RdvService
-http : HttpClient
-apiUrl : string
+getDemandesEnAttente(medecinId) : Observable<RdvDTO[]>
+getMesRdvs(patientId) : Observable<RdvDTO[]>
+getRdvsAvenir(patientId) : Observable<RdvDTO[]>
+getHistorique(patientId) : Observable<RdvDTO[]>
+accepterRdv(rdvId) : Observable<RdvDTO>
+refuserRdv(rdvId, motif) : Observable<RdvDTO>
+annulerRdv(rdvId, raison) : Observable<RdvDTO>
+reporterRdv(rdvId, nouvelleDate) : Observable<RdvDTO>
+verifierDisponibilite(creneauId) : Observable<boolean>
«Service»
NotificationSocketService
-socket : WebSocket
+connecter() : void
+ecouterNotifications() : Observable<NotificationDTO>
+deconnecter() : void
manipule
manipule
API REST
JSON / JWT
Models (Interfaces TypeScript)
«Interface»
RdvDTO
+id : number
+patientId : number
+medecinId : number
+date : string
+heureDebut : string
+heureFin : string
+duree : number
+typeConsultation : TypeConsultation
+statut : StatutRDV
+motifRefus : string
+raisonAnnulation : string
+lienTeleconsultation : string
«Interface»
NotificationDTO
+id : number
+destinataireId : number
+type : TypeNotification
+sujet : string
+message : string
+dateEnvoi : string
+estEnvoye : boolean
Backend NestJS [Controller → Service → Repository → Entity]
Controllers (Routes HTTP REST)
«Control»
RdvController
+GET /rdvs/medecin/:id → getDemandesEnAttente()
+GET /rdvs/patient/:id → getMesRdvs()
+GET /rdvs/patient/:id/avenir → getRdvsAvenir()
+GET /rdvs/patient/:id/historique → getHistorique()
+PATCH /rdvs/:id/accepter → accepterRdv()
+PATCH /rdvs/:id/refuser → refuserRdv()
+PATCH /rdvs/:id/annuler → annulerRdv()
+PATCH /rdvs/:id/reporter → reporterRdv()
+POST /rdvs/verifier-disponibilite → verifierDisponibilite()
délègue
WebSocket
temps réel
Services (Logique Métier)
«Service»
RdvService_Backend
+getDemandesEnAttente(medecinId) : Promise<RendezVous[]>
+getMesRdvs(patientId) : Promise<RendezVous[]>
+getRdvsAvenir(patientId) : Promise<RendezVous[]>
+getHistorique(patientId) : Promise<RendezVous[]>
+accepterRdv(rdvId) : Promise<RendezVous>
+refuserRdv(rdvId, motif) : Promise<RendezVous>
+annulerRdv(rdvId, raison) : Promise<RendezVous>
+reporterRdv(rdvId, nouvelleDate) : Promise<RendezVous>
+verifierDisponibilite(creneauId) : Promise<boolean>
+envoyerNotification(rdvId) : Promise<void>
appelle
«Service»
NotificationService
+creerNotification(destinataireId, type, sujet, message) : Promise<Notification>
+envoyerEmail(notification) : Promise<void>
+envoyerSMS(notification) : Promise<void>
+planifierRappel(rdvId) : Promise<void>
+broadcastWebSocket(notification) : void
accède
persiste
Repositories (Accès Données TypeORM)
«Repository»
NotificationRepository
+save(notification) : Promise<Notification>
+findByDestinataire(destinataireId) : Promise<Notification[]>
+markAsSent(id) : Promise<void>
«Repository»
RdvRepository
+findByMedecinEnAttente(medecinId) : Promise<RendezVous[]>
+findByPatient(patientId) : Promise<RendezVous[]>
+findUpcoming(patientId) : Promise<RendezVous[]>
+findHistorique(patientId) : Promise<RendezVous[]>
+updateStatut(rdvId, statut) : Promise<RendezVous>
+save(rdv) : Promise<RendezVous>
gère
gère
Entities (Modèles TypeORM / PostgreSQL)
«Entity»
Notification
+id : Long «PK»
+destinataireId : Long «FK»
+type : TypeNotification
+sujet : String
+message : String
+dateEnvoi : DateTime
+estEnvoye : Boolean
«Entity»
RendezVous
+id : Long «PK»
+patientId : Long «FK»
+medecinId : Long «FK»
+date : Date
+heureDebut : Time
+heureFin : Time
+duree : Integer
+typeConsultation : TypeConsultation
+statut : StatutRDV
+motifRefus : String
+raisonAnnulation : String
+dateAnnulation : DateTime
+lienTeleconsultation : String
assigné à
appartient à
«Enum»
TypeNotification
EMAIL
SMS
«Entity»
Medecin
+id : Long «PK»
+nom : String
+prenom : String
+email : String
+specialite : String
«Enum»
TypeConsultation
PHYSIQUE
TELE
«Enum»
StatutRDV
EN_ATTENTE
VALIDE
REFUSE
ANNULE
REPORTE
«Entity»
Patient
+id : Long «PK»
+nom : String
+prenom : String
+email : String
+telephone : String

***Figure 3.3 –Diagramme de classe Analytique* du cas "Gérer les rendez-vous"**



Le Tableau 3.5 présente le raffinement du cas "Gérer les rendez-vous" :

Cas	Gérer les rendez-vous
Acteur	-Patient -Médecin
Condition préalable	L'utilisateur est authentifié.
Condition résultante	Rendez-vous modifié ou annulé avec succès.
Description de scénario	- Le médecin reçoit une notification instantanée lors d'une demande de RDV. - Le médecin accepte ou refuse la demande en 1 clic (statut VALIDÉ / REFUSÉ). - Le patient consulte ses rendez-vous depuis son espace personnel. - Le patient peut annuler ou reporter un RDV depuis son espace. - La plateforme notifie les deux parties de tout changement.
Scénarii	- Annulation tardive : la plateforme affiche un avertissement. - Refus médecin : notification envoyée au patient.

Tableau 3.5 – Raffinement du cas « Gérer les rendez-vous »

3.4 Conception de Release 1

3.4.1 Conception de Sprint 0

Conception du cas "S'inscrire"

Nous trouvons dans cette figure 3.5 le diagramme de classe associé au cas "S'inscrire patient" :

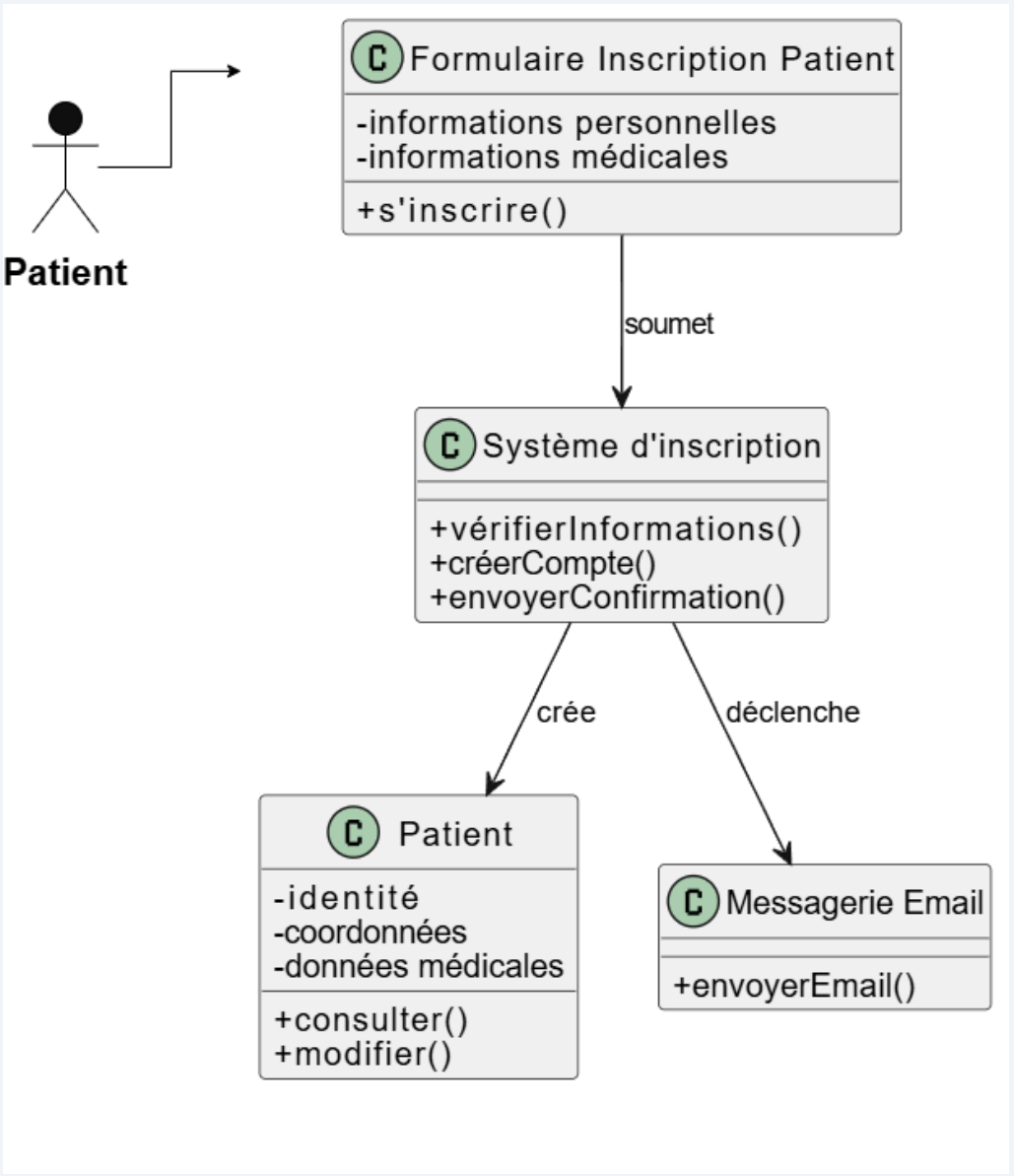


Figure 3.5 – Diagramme de classe conceptuel du cas « S'inscrire patient »

Dans la figure 3.6 nous pouvons observer le diagramme de classe du cas "S'inscrire Médecin" :

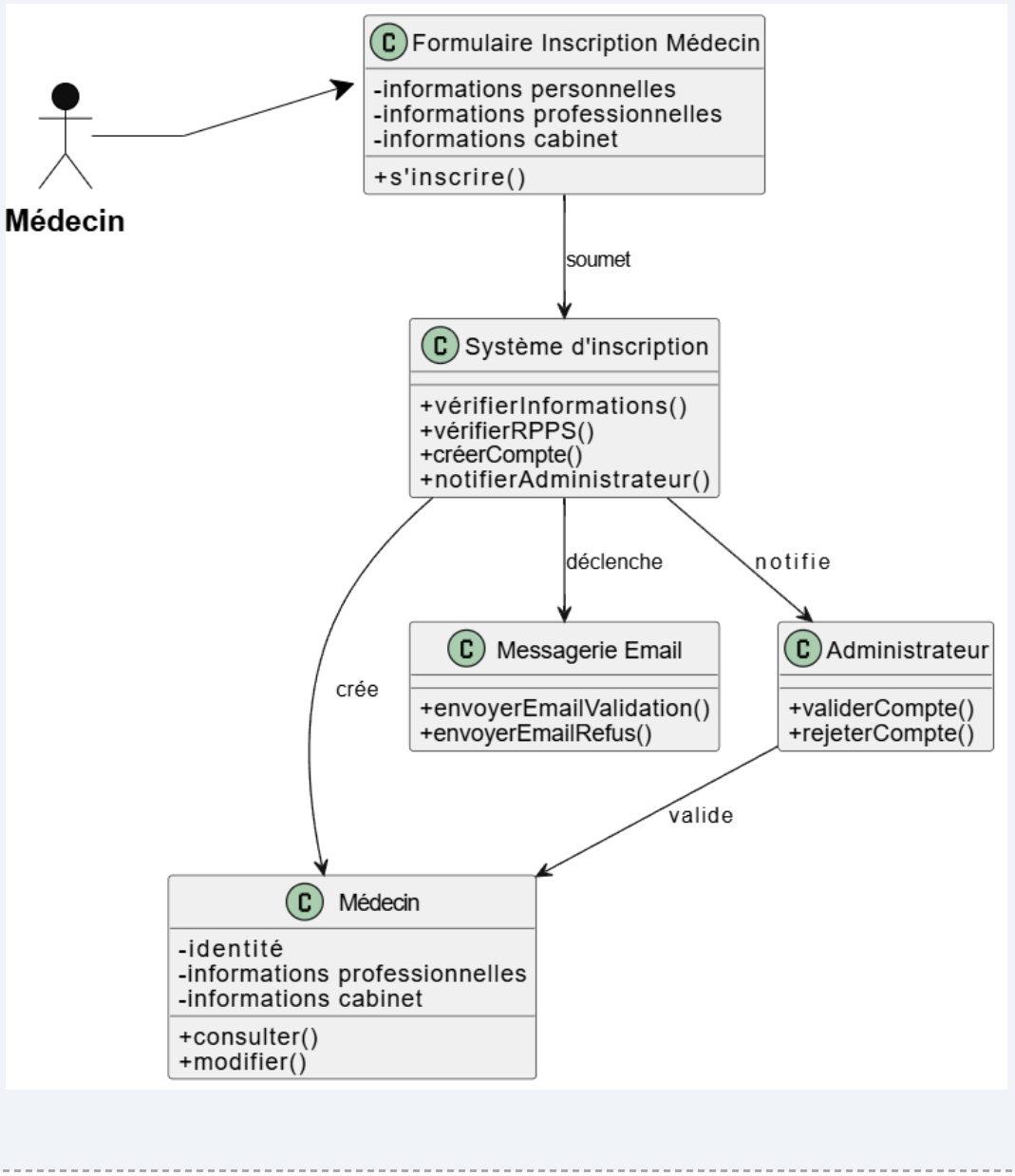


Figure 3.6 – Diagramme de classe conceptuel du cas d'utilisation « S'inscrire médecin »

Dans cette figure 3.7 on trouve le diagramme de séquence associé au cas "S'inscrire" :

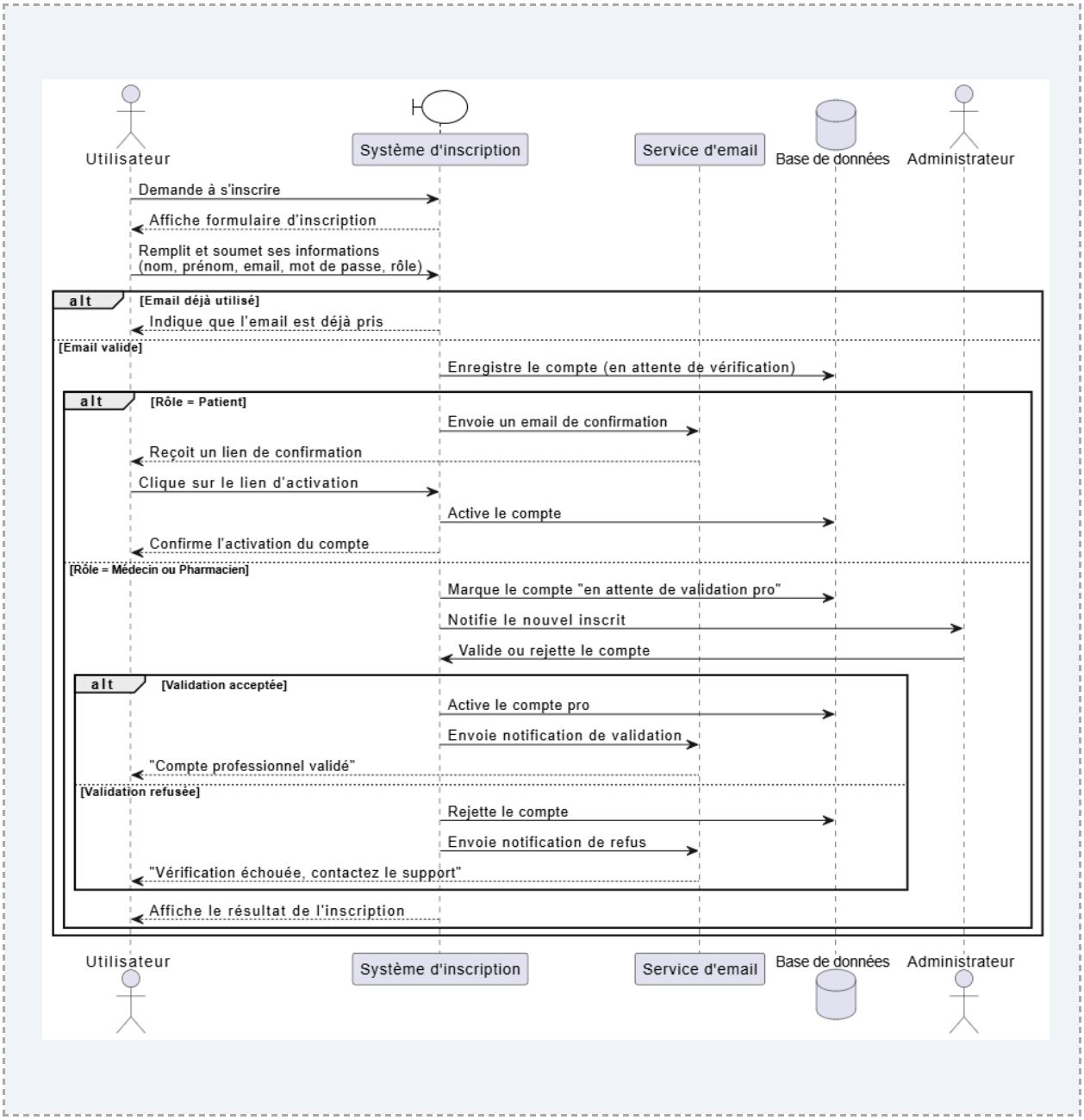


Figure 3.7 – Diagramme de séquence conceptuel du cas « S'inscrire »

Conception du cas "S'authentifier"

La figure 3.8 présente le diagramme de classe associé au cas "S'authentifier" :

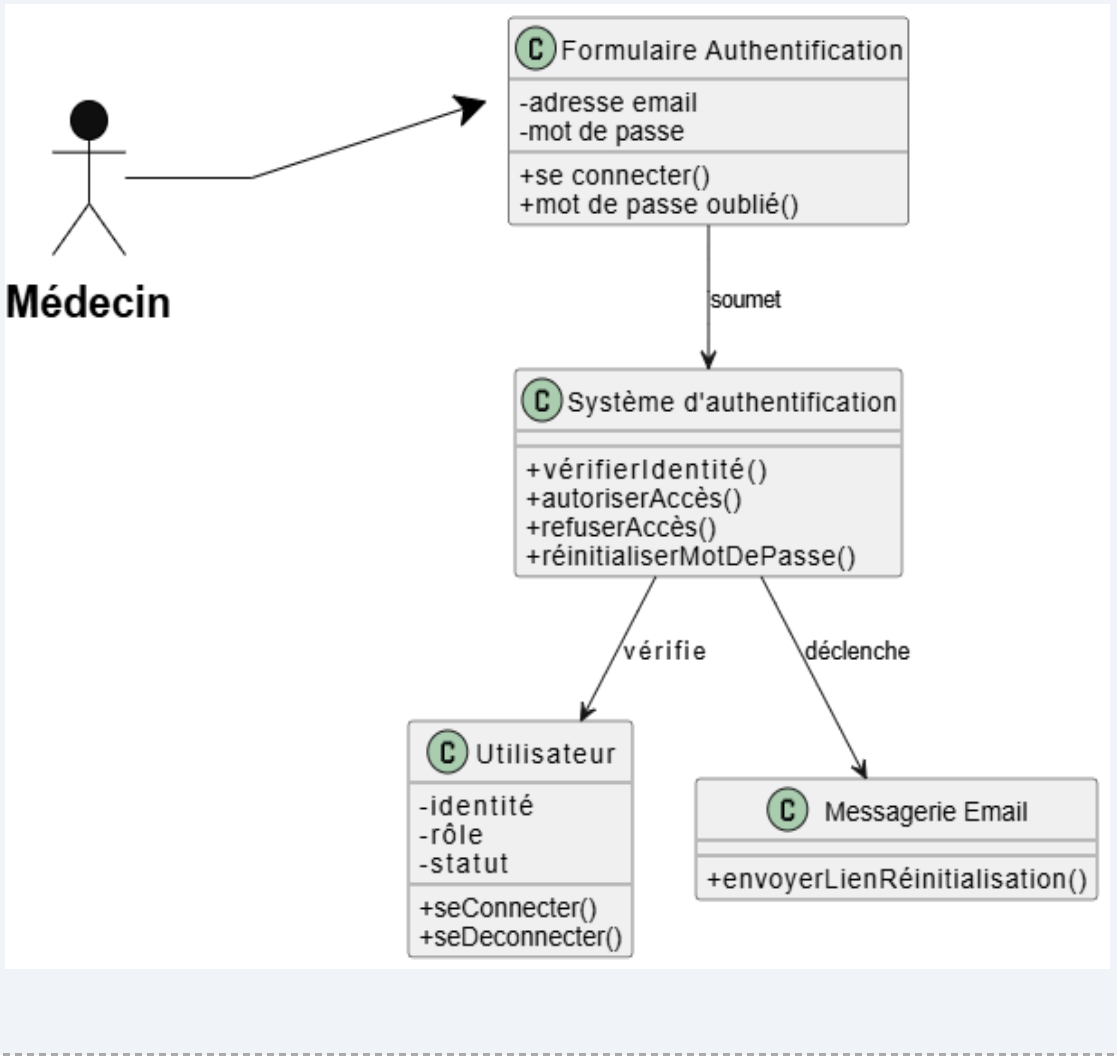


Figure 3.8 – Diagramme de classe Conceptuel du cas « S'authentifier »

La figure 3.9 présente le diagramme de séquence associé au cas "S'authentifier". Il illustre la séquence des messages échangés entre les différents objets du système :

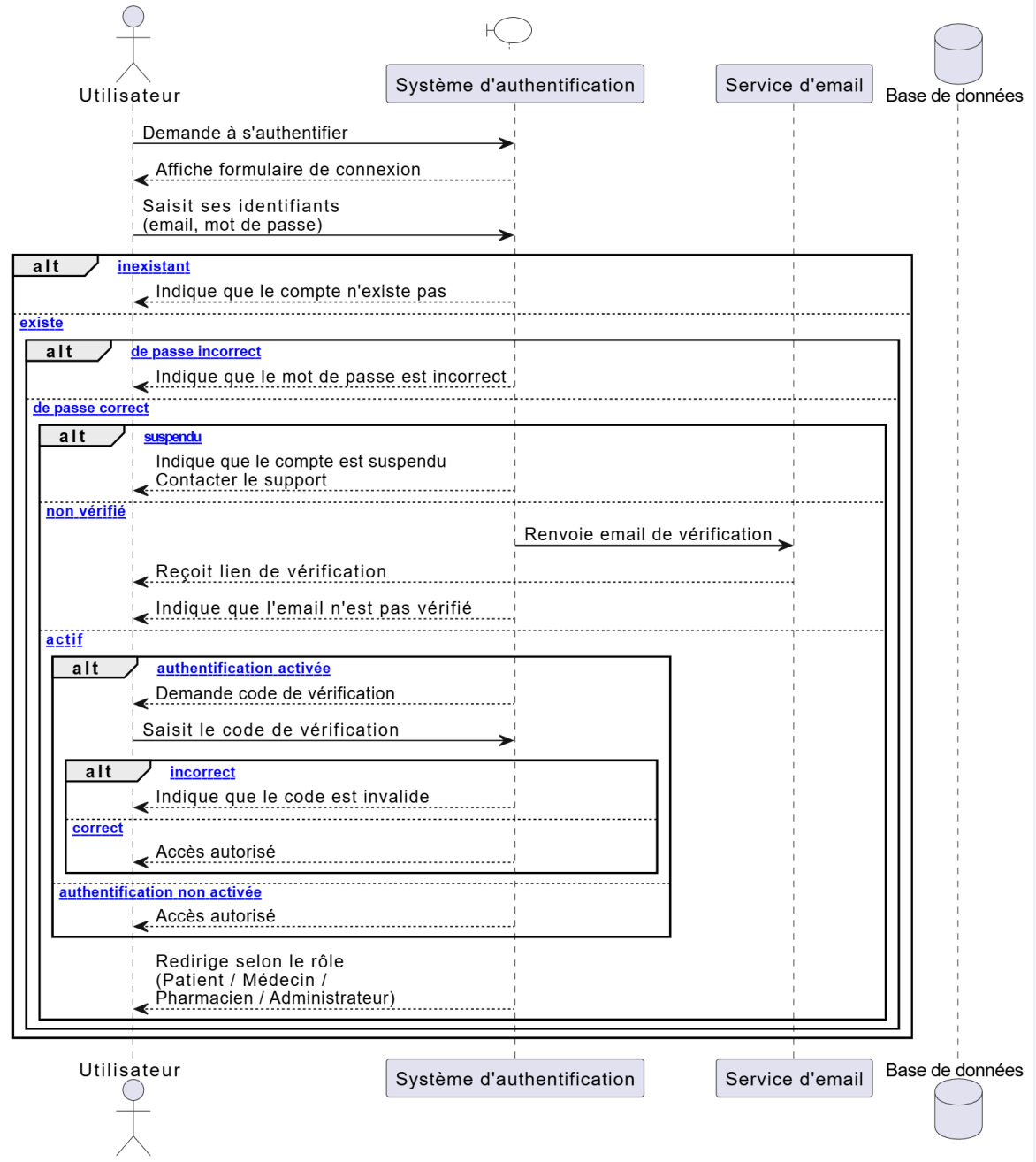


Figure 3.9 – Diagramme de séquence du cas « S'authentifier »

3.4.2 Conception de Sprint 1

Conception du cas "Prendre un rendez-vous"

Ce diagramme de classe 3.10 représente les différentes classes impliquées dans la prise de rendez-vous :

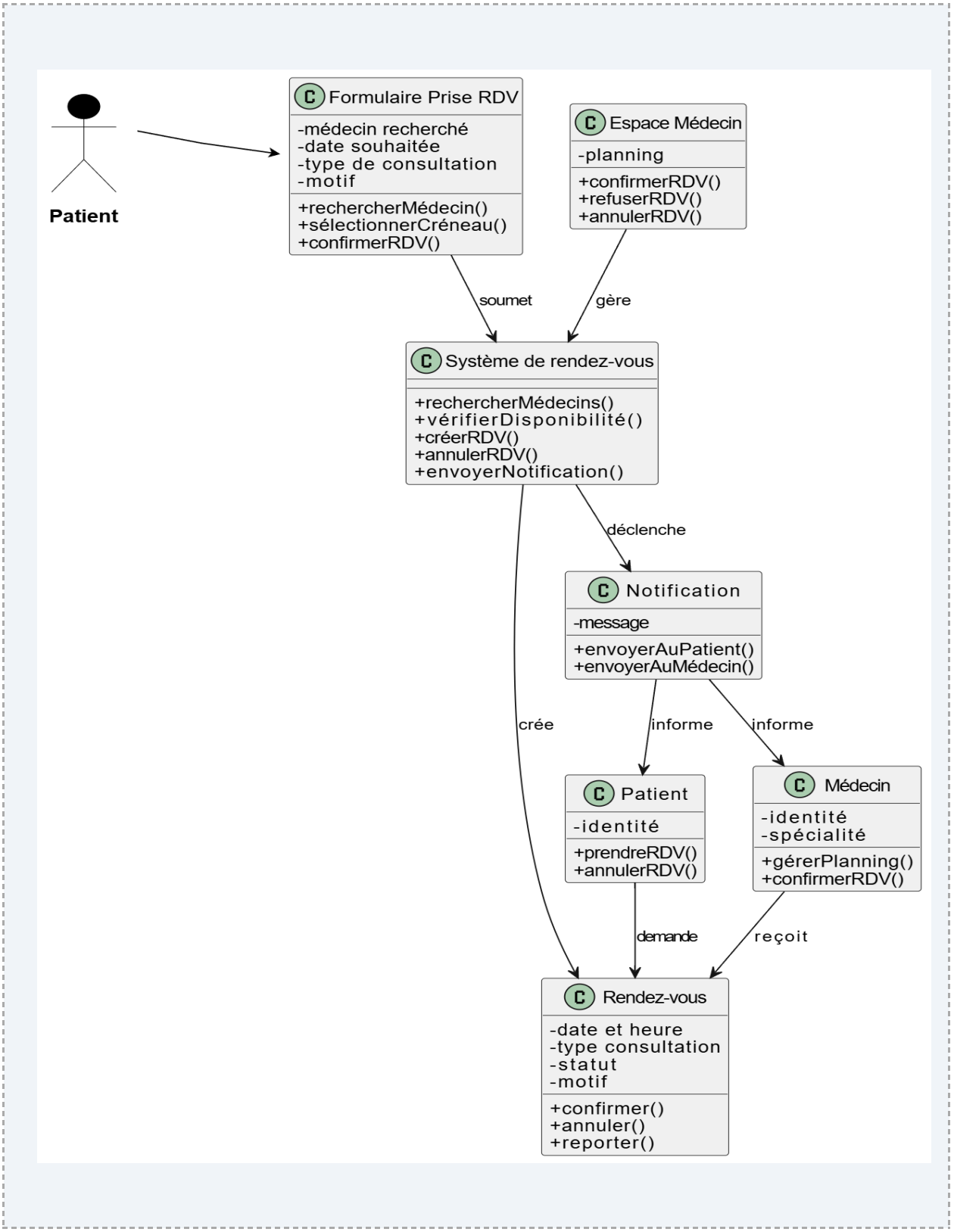


Figure 3.10 – Diagramme de classe Conceptuel du cas « Prendre un rendez-vous »

Ce diagramme de séquence 3.11 illustre la séquence des messages échangés entre les différents objets du cas "Prendre un rendez-vous" :

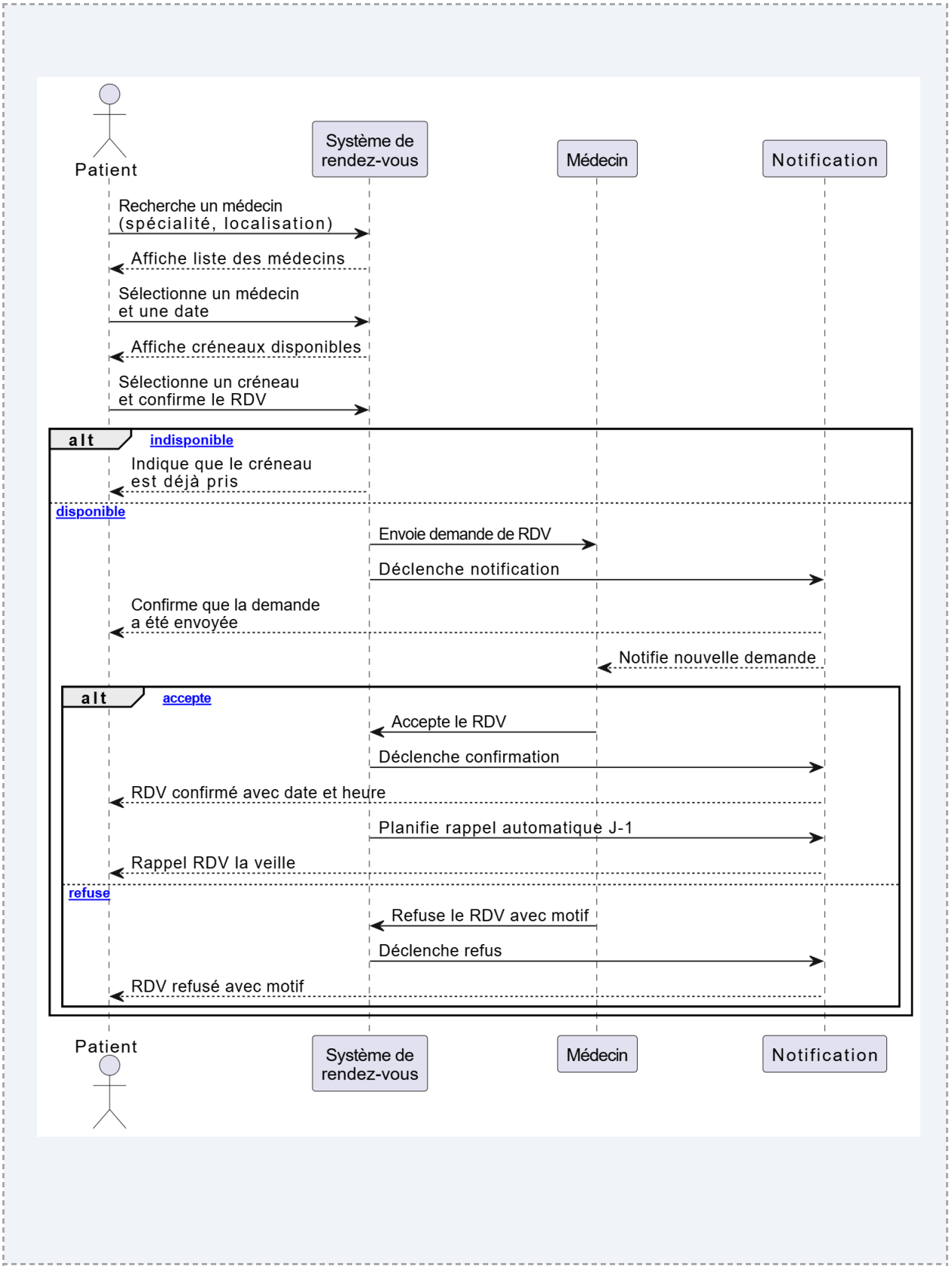


Figure 3.11 – Diagramme de séquence conceptuel du cas « Prendre un rendez-vous »

Conception du cas "Gérer les rendez-vous"

Ce diagramme de classe 3.12 illustre les différentes classes impliquées dans la gestion des rendez-vous :

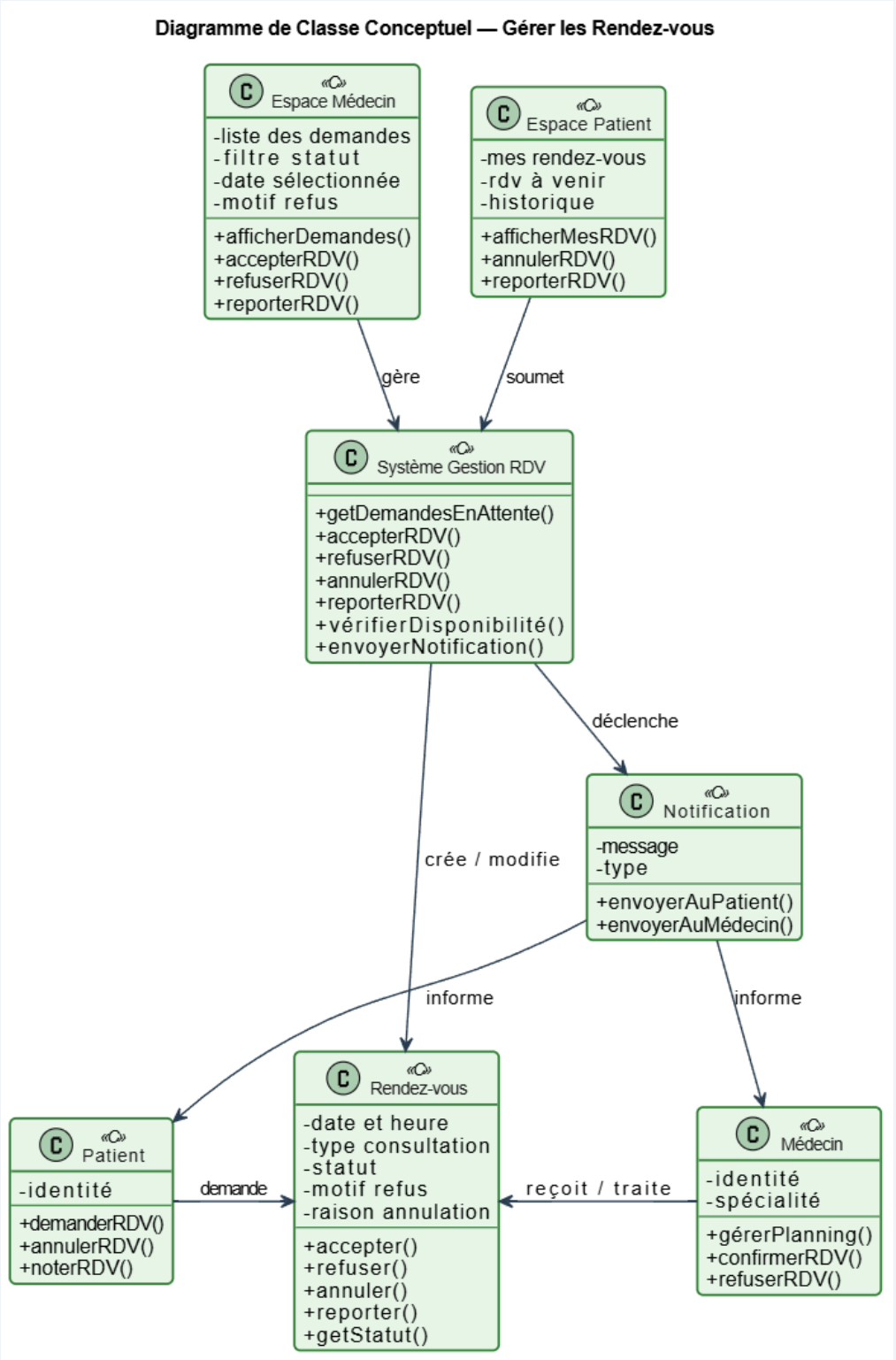


Figure 3.12 – Diagramme de classe conceptuel du cas « Gérer les rendez-vous »

Dans la figure 3.13, le diagramme de séquence du cas "Gérer les rendez-vous" illustre la séquence des interactions entre les différents acteurs et composants :

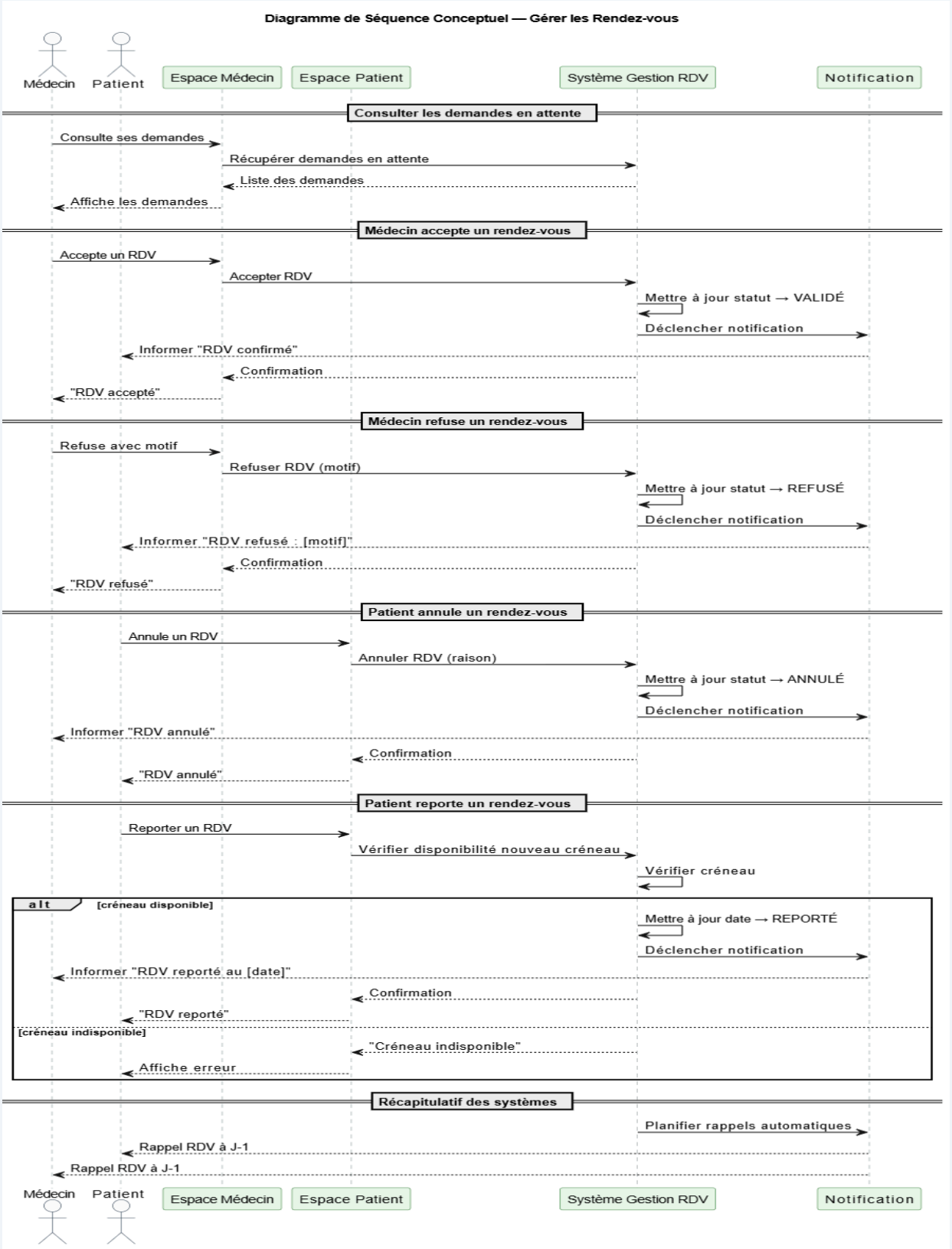


Figure 3.13 – Diagramme de séquence du cas « Gérer les rendez-vous »